1. Explain database schema with example.

Definition

A **Database Schema** is the logical structure or blueprint of the entire database. It defines how data is organized, the relationships among data items, and the structural constraints applied to them.

- It is designed during the initial phase of database creation and **rarely changes**.
- It is completely analogous to a variable declaration or a structural blueprint in programming languages.

Example

Consider a university database. The schema for a STUDENT table can be declared as:
Plaintext
STUDENT (Roll_No: Integer, Name: Varchar, Department: Varchar, GPA: Float)

This framework represents the schema, defining the attribute names, data types, and limitations without containing any actual student data records.

2. Define (i) Database (ii) DBMS

(i) Database

A **Database** is a structured, organized, and electronically stored collection of logically interrelated data. It serves as a central repository designed to allow efficient storage, retrieval, modification, and management of data.

- **Example:** A digital ledger containing all employee records, payroll info, and department allocations of an enterprise.

(ii) DBMS (Database Management System)

A **DBMS** is a specialized software application suite that acts as an interface between the database and its end-users or application programs. It provides mechanisms to create, read, update, delete (CRUD operations), and protect data.

- **Key functions:** Enforces security, ensures concurrency control, handles crashes, and preserves data integrity.
- **Examples:** Oracle, MySQL, PostgreSQL, Microsoft SQL Server.

3. Discuss five main advantages of database management system over file system.

Advantage	Description
1. Controlled Redundancy	Traditional file systems duplicate data across multiple files. A DBMS integrates data into a single logical structure, minimizing space waste and inconsistency risks.
2. Data Sharing & Concurrency	Multiple users can securely access the same data simultaneously without interfering with one another or causing data corruption.
3. Enforcement of Integrity	A DBMS allows developers to enforce constraints (e.g., Salary > 0 or Age >= 18). The system automatically rejects any inputs violating these rules.
4. Robust Backup & Recovery	Unlike manual copy-pasting in files, a DBMS features automated logging and recovery sub-systems to restore data to a consistent state after a power failure or system crash.

5. Enhanced Data Security	Access controls can restrict permissions down to specific tables, rows, or columns, ensuring users only see what they are explicitly authorized to view.

4. Explain the role of Database Administrator (DBA).

The **Database Administrator (DBA)** is a highly skilled technical professional or a team responsible for managing, securing, and maintaining the entire database environment.

Key Roles and Responsibilities

- **Schema Definition:** Designing the structural layout of the database (creating tables, views, and relationships).
- **Storage Structure and Access Strategy:** Deciding how files are physically stored on storage media and choosing appropriate indexing strategies for speed.
- **Security and Authorization:** Granting permissions to various users and regulating who has access to which segments of data.
- **Performance Tuning:** Monitoring database speed and optimizing slow queries to ensure low latency.
- **Backup and Recovery:** Formulating strict schedules for routine database backups and orchestrating recovery procedures in the event of system failures.

5. Explain different types of database end users.

Database users are categorized based on how they interact with the system:

- **1. Naive / Parametric Users:** Unsophisticated users who interact with the database through pre-written menu-driven or GUI-based application programs.
    - *Example:* An airline ticket agent or an ATM user.
- **2. Sophisticated Users:** Engineers, data scientists, and analysts who thoroughly understand the capabilities of the DBMS. They write direct, ad-hoc queries (like SQL commands) to extract insights without relying on external application interfaces.
- **3. Casual / Occasional Users:** Users who access the database infrequently but may need completely different information each time they access it.
    - *Example:* Middle or top management looking at monthly performance summaries.
- **4. Application Programmers:** IT professionals who write code (in Python, Java, C++, etc.) to build interactive application user interfaces that execute operations on the database underneath.

6. Explain database instance with example.

Definition

A **Database Instance** is the actual collection of information/data stored in the database at a **specific moment in time**.

- Unlike the schema (which remains static), the database instance is **dynamic** and changes constantly whenever data is inserted, updated, or deleted.
- It is completely analogous to the current *value* of a variable at a specific line during runtime.

Example

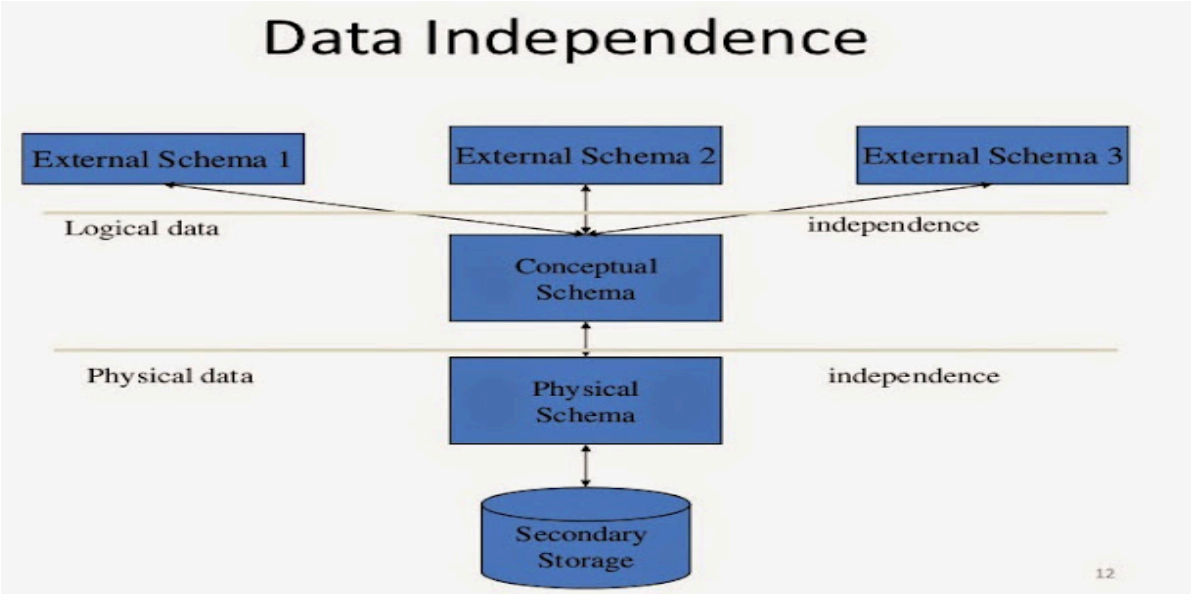
Suppose a COURSE table has the following state at **09:00 AM**:

Course_ID	Course_Name
CS101	DBMS
CS102	Data Structures

If an instructor inserts a new course (CS103: Computer Architecture) at **09:05 AM**, the table updates. The state at 09:00 AM represents one instance, and the state at 09:05 AM represents a completely new database instance.

7. Explain logical data independence and physical data independence with suitable diagram.

Data independence represents the capacity to modify the schema at one level of the database architecture without forcing modifications at higher levels.



Physical Data Independence

The capacity to alter the physical schema without altering the conceptual schema. Changes to storage devices, indexing techniques (B+ trees, hashing), or file locations do not require updates to your database tables or application code.

Logical Data Independence

The capacity to alter the conceptual schema without altering the external views or existing application programs. You can add a new column, modify a data constraint, or split a table without breaking the user interfaces that depend on the overall database structure.

8. What is the purpose of database system? Describe the three levels of data abstraction.

Purpose of a Database System

The fundamental goal of a database system is to provide users with an **abstract view of data**. It hides the complex physical storage and maintenance details behind simple, high-level interfaces, eliminating data isolation, inconsistency, and security weaknesses common in traditional file systems.

Three Levels of Data Abstraction (ANSI-SPARC Architecture)

- **1. Physical Level (Lowest Level):** Describes *how* the data is actually stored in storage devices. It deals with blocks, bytes, memory addresses, and file structures.
- **2. Conceptual/Logical Level (Middle Level):** Describes *what* data is stored in the database and *what relationships* exist among those data points. It focuses on tables, columns, data types, and integrity constraints.
- **3. View/External Level (Highest Level):** Describes only a specific part of the total database that is relevant to a particular user or application. It abstracts away all other unrelated tables for simplicity and security.

9. What are the different types of schema?

Based on the standard 3-schema architecture, database systems maintain three core schema divisions:

- **1. Physical Schema:** Defines the physical organization and low-level storage layout of the database on disk. It outlines block sizes, clusters, file allocations, and index pathways.
- **2. Conceptual Schema:** Defines the logical design, entities, attributes, relationships, and data constraints for the entire organization's data. It represents the master blueprint of the data structure.
- **3. External Schema (View Schema):** Represents various tailor-made database schemas targeted at specific groups of end-users. A database can have multiple external schemas, showing only the records a specific user group is authorized to view.

10. Explain the database design process in detail.

The database design process is a structured methodology involving several critical phases:
Plaintext
Requirements Analysis ➔ Conceptual Design (ERD) ➔ Logical Design ➔ Schema Refinement ➔ Physical Design ➔ Implementation

- **Step 1: Requirements Analysis:** Designers interview stakeholders to gather, record, and analyze data needs, functional rules, and transaction volumes.
- **Step 2: Conceptual Schema Design:** The requirements are translated into a high-level, conceptual data model. This phase typically produces an **Entity-Relationship Diagram (ERD)** identifying major entities and their relationships.
- **Step 3: Logical Database Design:** The abstract conceptual model (ERD) is mapped directly into the specific structural model supported by the target DBMS (typically transforming entities and relationships into **Relational Tables**).
- **Step 4: Schema Refinement (Normalization):** The generated tables are analyzed for redundancies and anomalies. Designers use normalization rules (1NF, 2NF, 3NF, BCNF) to decompose tables for clean data storage.
- **Step 5: Physical Schema Design:** Specifying storage configurations, internal file placements, cluster methods, and indexing choices to ensure high performance.
- **Step 6: Implementation:** Writing actual DDL scripts (CREATE TABLE etc.) to instantiate the structures, set security access rules, and load data.

11. Explain the difference between meta data and data dictionary with example. What are the different data models?

Metadata vs. Data Dictionary

Concept	Definition	Example
Metadata	**"Data about data."** It represents structural information describing properties of primary data elements.	The fact that a column named Roll_No is of type INT and is a PRIMARY KEY.
Data Dictionary	The **centralized software repository** that explicitly *stores* all accumulated metadata within the DBMS.	System catalogs/tables like INFORMATION_SCHEMA.TABLES or sys.objects in SQL.

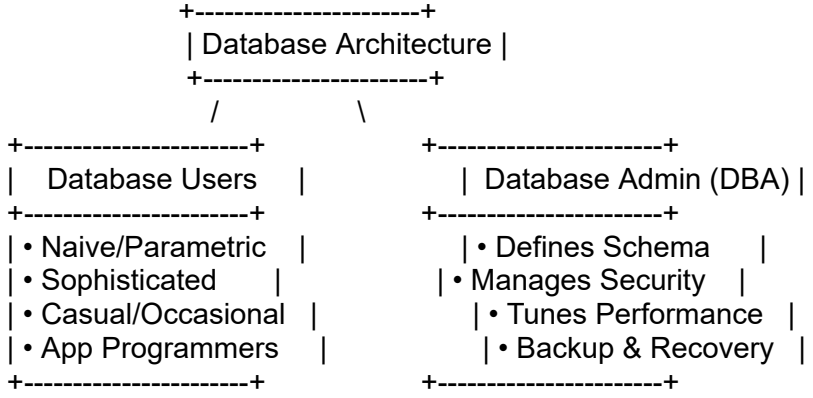
Different Data Models

A data model describes the structural framework used to process and organize data.

1. **Relational Model:** Data is organized in tabular grids of rows and columns (SQL databases).
2. **Entity-Relationship Model (ER Model):** Conceptual framework representing data as real-world entities and associations.
3. **Object-Oriented Data Model:** Data is maintained as code objects incorporating attributes and methods together.
4. **Semi-Structured Data Model:** Data formats like JSON or XML where structure is embedded with the data.
5. **Legacy Models:** Hierarchical Model (tree-based) and Network Model (graph-based).

12. Discuss about Database users and Administrators.

An engineering presentation for this combined question should clearly show the dichotomy of roles within the system:



- **Database Users:** These are individuals who interact with the system to fulfill organizational business processes. They do not maintain the system; they simply consume or input information using tailored levels of abstraction.
- **Database Administrators (DBA):** This role possesses absolute administrative privileges. The DBA handles structural tuning, provisions authorization privileges to legitimate users, updates underlying schemas, protects integrity constraints, and guarantees maximum up-time.

13. Explain about Database languages with examples.

A DBMS provides specialized subset languages to process operations at different structural levels:

- **1. Data Definition Language (DDL):** Used to build, modify, or eliminate the underlying structural database schemas and definitions.
  - *Examples:* CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE.
- **2. Data Manipulation Language (DML):** Used to access, modify, insert, or clean up actual data records populated within the predefined schemas.
  - *Examples:* SELECT, INSERT, UPDATE, DELETE.
- **3. Data Control Language (DCL):** Used by database administrators to enforce security profiles and manage individual user access permissions.
  - *Examples:* GRANT (give access privileges), REVOKE (take back access privileges).
- **4. Transaction Control Language (TCL):** Used to monitor and manage the execution flow of data modification transactions to ensure system safety.
  - *Examples:* COMMIT (save changes permanently), ROLLBACK (undo modifications if an error occurs), SAVEPOINT.

10 marks

1. Discuss Three Tier Database Architecture with diagram .

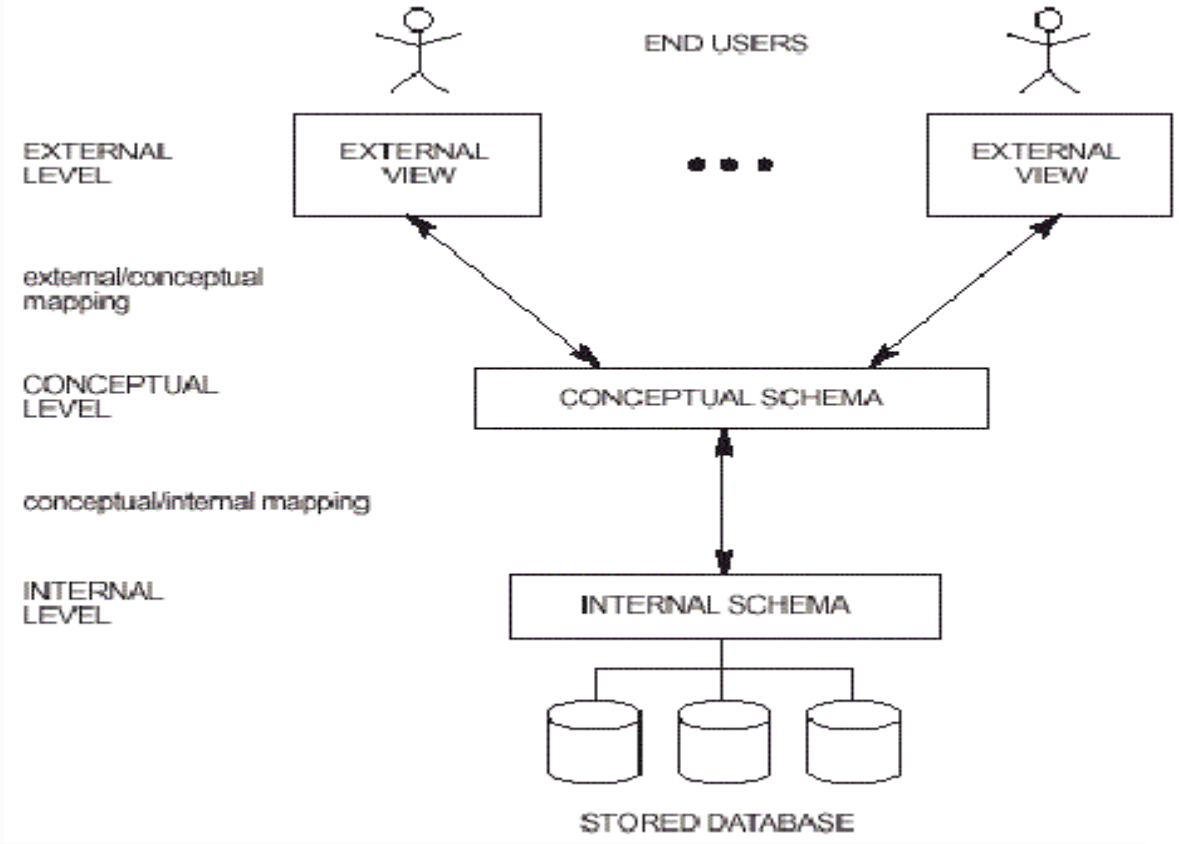
DATABASE SYSTEM ARCHITECTURE

Three layers of architecture of DBMS:-

1. External level/View level-It is the user view and it shows the data that is relevant to the user. It deals with the way in which individual user view the data.It hides the details of internal and conceptual level.

2. Conceptual level/Logical level-It describes what data is stored in the database and the relationship among them.It contains the logical structure of the database as seen by the DBA.

3. Internal level/Physical level- It is the physical representation of the database. It covers the data structures used to store the data on storage devices,It also deals with storage space allocation.



OBJECTIVES OF 3 LEVEL ARCHITECTURE-

1. Each user access the same data but have a different view.
2. No dealing with physical storage details.
3. DBA changes database storage structure without effecting users view.
4. The main objective of this 3 level architecture is to provide **DATA INDEPENDENCE.**

2. Define the terms: 1) physical schema, 2) logical schema, 3) Conceptual Schema with suitable diagram.

A database schema represents the structural design of the database. The three types of schemas correspond directly to the levels of data abstraction in a database system.

1) Physical Schema: The physical schema represents the design of the database at its lowest, physical level. It dictates the actual storage layout on hardware components.

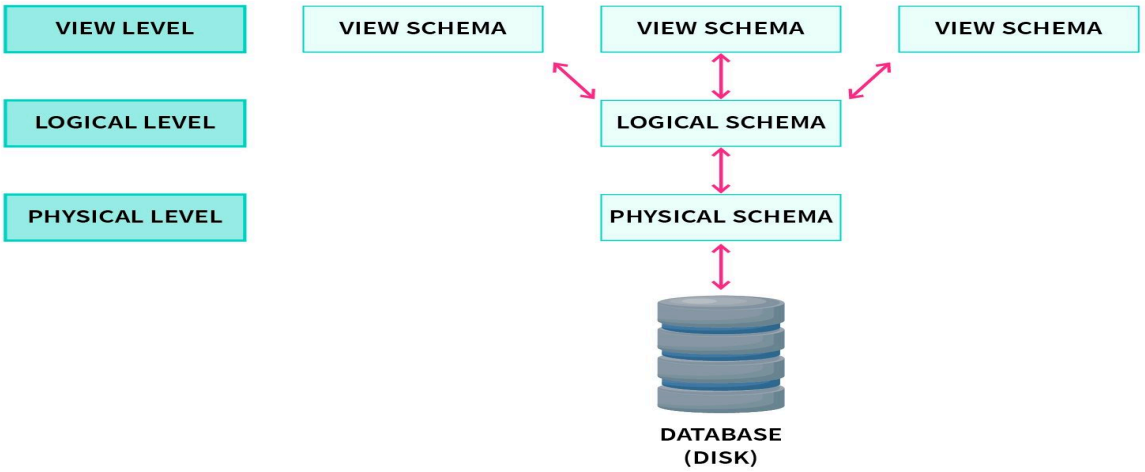
- **Details:** It specifies block sizes, file allocation techniques, record indexing (such as B+ trees), data compression, and encryption details.

2) Logical Schema: The logical schema is the representation of data in a format that maps directly to the implementation structure of the target DBMS (such as the relational model).

- **Details:** It represents the conceptual schema translated into actual tables, primary keys, foreign keys, columns, and data types (e.g., INT, VARCHAR).

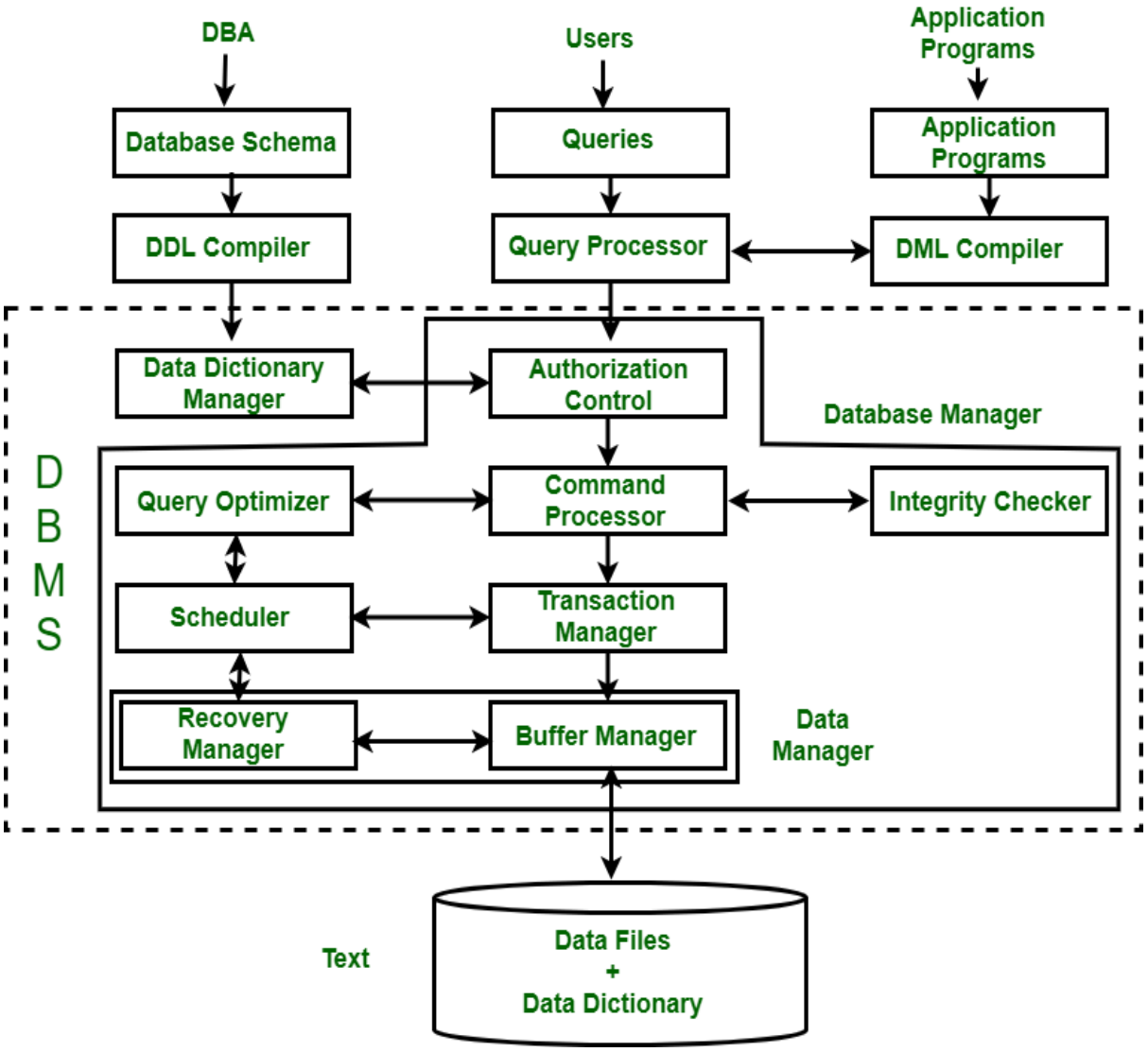
3) Conceptual Schema: The conceptual schema is a high-level, abstract design of the entire database independent of any specific hardware or software platform.

- **Details:** It models the entire business environment by identifying core entities, their attributes, and relationships (typically represented using an Entity-Relationship Diagram during the design phase).



3. Draw the Architecture of Database.

Database Architecture refers to the internal components of the DBMS, including the Query Processor, Storage Manager, and Disk Storage. It also defines the interaction of these components.



4. What are the two types of participation constraint?

What are different access types in DML?

Participation constraints in an ER Diagram specify whether the existence of an entity depends on its being related to another entity via a relationship set.

- **1. Total Participation (Dependency):**
  - Occurs when **every entity** in the entity set must participate in at least one relationship instance within the relationship set.
  - *Example:* In a company database, every `EMPLOYEE` must work for a `DEPARTMENT`. An employee cannot exist in the system without an assigned department.
  - *ERD Notation:* Represented using a **double line** connecting the entity to the relationship.
- **2. Partial Participation:**
  - Occurs when **only some** of the entities in the entity set participate in the relationship instances, while others do not.
  - *Example:* Not every `EMPLOYEE` manages a `DEPARTMENT`. Only a few select employees act as managers.
  - *ERD Notation:* Represented using a **single line** connecting the entity to the relationship.

Part 2: Access Types in DML (Data Manipulation Language)

DML commands interact with data values in the database. The interaction falls into two primary access types:

- **1. Read-Only Access (Retrieval / Querying):**
  - The user requests existing data from the database without altering the underlying records.
  - *Primary Command:* `SELECT`.
- **2. Read-Write Access (Modification / Update):**
  - The user explicitly alters the contents of the database through data insertion, deletion, or value modifications.
  - *Primary Commands:* * `INSERT` (Adds new tuples/rows).
    - `UPDATE` (Modifies values in existing tuples).
    - `DELETE` (Removes existing tuples).

5. Differentiate File Processing System with Database Management System.

Feature / Dimension	File Processing System	Database Management System (DBMS)
Data Redundancy & Inconsistency PDF	High. Different applications maintain separate files, leading to duplicated data and conflicting records. PDF	Low. Data is integrated into a unified structure, minimizing redundancy and preventing inconsistency. PDF
Data Independence	Absent. File structures are hardcoded into application programs; changing a file format requires rewriting code.	High. Clear separation between physical storage layouts and logical schemas (Data Independence).
Data Access & Retrieval	Inefficient. Requires writing a new custom program in languages like C/Java to answer any new ad-hoc query.	Efficient. Allows users to write quick, ad-hoc queries using high-level declarative languages like SQL.
Concurrency Control	Poor. If two users edit the same file simultaneously, one set of updates will overwrite the other, causing data loss.	Robust. Uses locking mechanisms and transaction protocols to support thousands of concurrent users safely.
Security & Access Control	Weak. Access control is limited to basic file-level OS permissions (Read/Write/Execute).	Granular. Security policies can restrict access down to specific tables, rows, or individual columns for different users.
Integrity Constraints	Difficult to enforce. Validation logic must be manually written inside every application program.	Centralized enforcement. Constraints (e.g., `NOT NULL`, `FOREIGN KEY`) are defined in the schema and enforced by the system.
Backup and Recovery	Manual/Rudimentary. If the system crashes during a file transfer, data can be corrupted or completely lost.	Automated. Utilizes transaction logging and recovery modules to restore data to its last consistent state after a crash.

Cost & Complexity	Low initial cost. Uses standard operating system tools and files.	High initial cost. Requires purchasing specialized software licenses, training staff, and using robust hardware.

MODULE 2

1. Explain Derived attribute and Multivalued attribute with example and diagram.

Derived Attribute

- **Definition:** An attribute whose value is not physically stored in the database but is instead calculated dynamically from other related attributes or system values.
- **ERD Notation:** Represented by a **dashed oval**.
- **Example:** Age is a derived attribute because it can be calculated automatically by subtracting a person's Date_of_Birth from the current system date.

Multivalued Attribute

- **Definition:** An attribute that can hold more than one value for a single entity instance.
- **ERD Notation:** Represented by a **double oval**.
- **Example:** An employee can have multiple phone numbers (Phone_No) or multiple academic degrees (Skills).

2. What is Key Attribute? Explain with example.

Key Attribute

- **Definition:** A key attribute is an attribute (or a combination of attributes) whose values uniquely identify each individual entity instance within an entity set. No two distinct entities in the set can share identical values for a key attribute.
- **ERD Notation:** Represented by an oval with the attribute's name **underlined**.
- **Key Characteristics:** * It must contain unique values.
    - It cannot contain NULL values.

Example

In a university management system, consider the STUDENT entity set:

- Attributes include Roll_No, Name, Email, and Address.
- While multiple students can share the same name or address, each student is assigned a completely unique Roll_No.
- Therefore, Roll_No serves as the **Key Attribute** for the STUDENT entity set.

3. What is foreign key? Explain with example.

Foreign Key

- **Definition:** A foreign key is an attribute or a collection of attributes in a relational database table that refers to the primary key of another table (or sometimes the same table).
- **Purpose:** It establishes and enforces a logical link or relationship between the data across two distinct tables, thereby ensuring **Referential Integrity**.

Example

Consider two database tables: DEPARTMENT and EMPLOYEE.

1. DEPARTMENT Table (Parent/Referenced Table)

- Dept_ID (Primary Key)
- Dept_Name

2. EMPLOYEE Table (Child/Referencing Table)

- Emp_ID (Primary Key)
- Emp_Name
- Dept_ID (**Foreign Key** pointing to DEPARTMENT.Dept_ID)

4. Explain Hierarchical Data Model with example.

Hierarchical Data Model

- **Definition:** One of the oldest database models, where data is organized into a tree-like structure.
- **Structure:** * Data is represented as a collection of records connected via links.
    - It follows a **Top-Down** configuration beginning with a single **Root Node**.

- It enforces a strict **1:N (One-to-Many) relationship**, meaning each child record can have only *one* parent record, but a parent record can have *multiple* child records.

Example

Consider a university organizational hierarchy:

- **University (Root)**
  - **Department (Child of University)**
    - **Course (Child of Department)**
    - **Professor (Child of Department)**
  - **Infrastructure (Child of University)**

5. Define Entity, Attributes, Entity set, relationship with appropriate notations in ERD. Draw the notation for multivalued attributes. Give one example.

Definitions and Notations

1. **Entity:** A real-world object or concept that is distinguishable from other objects (e.g., a specific student). **Notation: Rectangle**.
2. **Attribute:** A descriptive property or characteristic belonging to an entity (e.g., Student Name). **Notation: Oval**.
3. **Entity Set:** A collection of entities of the same type that share the same properties (e.g., all students in a college). **Notation: Rectangle** (labeled with a plural noun).
4. **Relationship:** An association or link among several entities (e.g., Student *Enrolls in* a Course). **Notation: Diamond**.
5. **Multivalued Attribute Notation:** Represented using a **Double Oval**.

Example

An ERD segment capturing a college interaction:

- **Entity Sets:** STUDENT and COURSE.
- **Relationship:** ENROLLS_IN.
- **Attributes for STUDENT:** Roll_No (Key), Name (Simple), and Phone_No (Multivalued - drawn with a **Double Oval**).

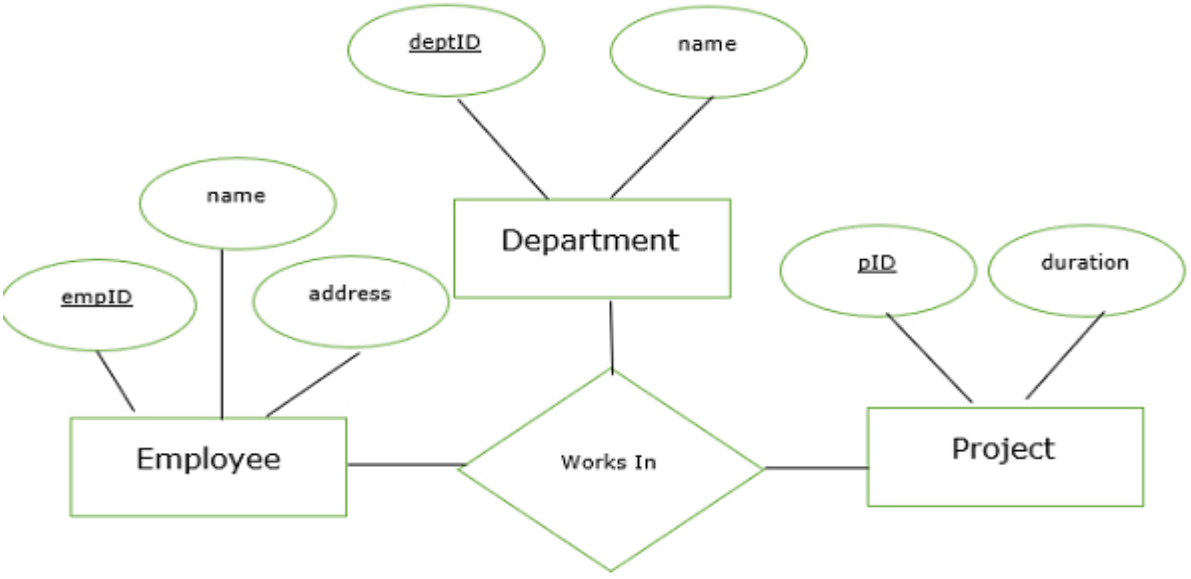
6. Draw ER diagram for Ternary Relationship set with suitable example.

Ternary Relationship

- **Definition:** A ternary relationship is a relationship set that involves exactly **three distinct entity sets** simultaneously. It is used when a binary (two-entity) relationship is insufficient to capture the real-world operational rules accurately.

Example: Consultant, client and contract are three different entities with different attributes.

These three entities are related with a single relationship called "signs".



Here,

- Three foreign keys of works in the table are empID, deptID, pID.
- They refer to the primary key of an employee, department, project.
- These attributes are the component of the primary key of works in table.
- The primary key works in the table is (empID,depID,pID).

7. What does the cardinality ratio specify? List any eight applications of DBMS.

Cardinality Ratio

The cardinality ratio specifies the maximum number of relationship instances that an entity can participate in. It defines the structural constraints between mapping entities. The four types are:

1. **One-to-One (1:1):** An entity in A is associated with at most one entity in B, and vice-versa.
2. **One-to-Many (1:N):** An entity in A is associated with any number of entities in B, but B can associate with at most one in A.
3. **Many-to-One (N:1):** Multiple entities in A can associate with one entity in B.
4. **Many-to-Many (M:N):** Multiple entities in A can associate with multiple entities in B.

Eight Applications of DBMS

1. **Banking Systems:** For tracking customer accounts, deposits, loans, and daily financial transactions.
2. **Airlines & Railways:** For handling flight/train schedules, passenger routing, and online ticket reservations.
3. **Universities & Schools:** For maintaining student registrations, grading sheets, course schedules, and fee records.
4. **Telecommunications:** For storing call logs, customer billing data, network configurations, and prepaid balances.
5. **E-commerce/Sales:** For managing product inventories, customer shopping carts, invoices, and tracking shipments.
6. **Human Resources (HR):** For administering employee payrolls, tax deductions, performance evaluations, and leave balances.
7. **Manufacturing & Logistics:** For monitoring supply chains, warehouse stock levels, and tracking assembly outputs.
8. **Social Media Platforms:** For mapping user profiles, complex friend networks, posts, comments, and media uploads.

8. Define Relational Database. Explain data representation in RDBMS.

Definition of Relational Database

A **Relational Database** is a digital collection of data points organized into formally described, interconnected tables (relations). It is based on the relational data model introduced by E.F. Codd, where structural operations are governed by mathematical relational algebra.

Data Representation in RDBMS

1. **Tables (Relations):** The logical container representing a specific entity set (e.g., STUDENT table).
2. **Rows (Tuples / Records):** Each individual horizontal row represents a single, complete data record or instance within that entity set.
3. **Columns (Attributes / Fields):** Each vertical column denotes a distinct structural characteristic or property of the entity (e.g., Roll_No, Course).
4. **Domain:** The set of allowable, atomic values assigned to a particular column (e.g., Roll_No must be a positive integer).

9. What is referential integrity constraints? How it is related to foreign key. Explain with example.

Referential Integrity Constraints

- **Definition:** A data integrity rule ensuring that associations between tables remain valid and consistent. It dictates that a value in one table pointing to another table **must match an existing valid record** in that referenced table.

Relationship to Foreign Key

The **Foreign Key** is the active structural mechanism used to *implement and enforce* the referential integrity constraint inside an RDBMS. When a column is designated as a foreign key, the RDBMS automatically verifies every insert, update, and delete action to prevent broken links.

Example

Consider a system tracking student library assignments:

- **BOOKS Table (Parent)** -> Primary Key = Book_ID [Values: B01, B02]
- **ISSUED Table (Child)** -> Foreign Key = Book_ID referencing BOOKS(Book_ID)

10 MARKS

1. Define Weak Entity set, candidate key, primary key and foreign key.

Weak Entity Set

- **Definition:** An entity set that does not possess sufficient attributes to form a primary key on its own.
- **Characteristics:** It depends on a dominant entity set (called the identifying or owner entity set) for its existence.
- **Discriminator:** It uses a partial key (discriminator), underlined with a **dashed line**, to uniquely distinguish weak entities belonging to the same owner entity.
- **Example:** A DEPENDENT table (with attributes Name, Age) cannot exist without an EMPLOYEE table.

Candidate Key

- **Definition:** A minimal superkey—a single attribute or a minimal set of attributes that can uniquely identify each record in a relation without any redundancy.
- **Properties:** It must contain unique values and cannot be NULL. A table can have multiple candidate keys.
- **Example:** In an EMPLOYEE table, both SSN and Email are candidate keys because either can uniquely identify a worker.

Primary Key

- **Definition:** The specific candidate key chosen by the database designer to uniquely identify tuples within a relation.
- **Properties:** It must be unique, cannot accept NULL values, and there can only be **one primary key per table**.
- **Example:** If SSN is chosen out of the candidate keys to organize the EMPLOYEE table, it becomes the primary key.

Foreign Key

- **Definition:** An attribute or a set of attributes in a table that references the primary key of another table.
- **Purpose:** It establishes a link between data in two tables and enforces referential integrity.
- **Example:** A Dept_No attribute inside an EMPLOYEE table referencing the primary key Dept_No of a DEPARTMENT table.

2. Explain Generalization and Specialization with example.

Specialization and **Generalization** are structural high-level data modeling concepts used to manage complexity in an Entity-Relationship Diagram (ERD) by establishing structural hierarchies.

1. Specialization

- **Approach: Top-Down** design process.
- **Explanation:** It starts with a single, high-level comprehensive entity set and breaks it down into lower-level, highly specific sub-entity sets based on distinct characteristics.
- **Process:** Attributes and relationships specific to a subset are isolated and assigned exclusively to the lower-level sub-entities.

- **Example:** Consider a generalized entity set ACCOUNT. Based on the specific operational traits of bank accounts, it can be specialized downwards into two sub-entity sets: SAVINGS_ACCOUNT (which gains a unique Interest_Rate attribute) and CURRENT_ACCOUNT (which gains a unique Overdraft_Limit attribute).

2. Generalization

- **Approach: Bottom-Up** design process.
- **Explanation:** It takes multiple structural lower-level entity sets that share common properties and synthesizes them into a single, high-level generalized super-entity set.
- **Process:** Common attributes are moved up to the super-entity to eliminate design redundancy.
- **Example:** Suppose a database has separate entities named CAR (attributes: Reg_No, Price, Seating_Capacity), TRUCK (attributes: Reg_No, Price, Cargo_Capacity), and BIKE (attributes: Reg_No, Price, Engine_CC). Through generalization, the shared attributes (Reg_No, Price) are extracted to form a high-level super-entity named VEHICLE, while the unique fields remain attached to the sub-entities below.

3. What is the difference between Hierarchical and Network Model? Explain with example.

Key Structural Differences

Comparison Dimension	Hierarchical Data Model PDF	Network Data Model PDF
Structural Layout	Organizes data in an inverted tree-like, top-down structure.	Organizes data in an interconnected graph-like network structure.
Relationship Mapping	Strictly supports **1:N (One-to-Many)** relationships.	Supports **M:N (Many-to-Many)** as well as 1:N relationships directly.
Parent-Child Rule	A child record can have **only one** parent record.	A child record can have **multiple parent records** (multiple owners).
Data Access Pathway	Restricted. Navigation must start from the root node and trace downward along defined paths.	Flexible. Data points can be accessed through multiple paths since nodes are linked via a mesh network.
Anomalies	Deleting a parent node automatically deletes all its associated child nodes (high risk of accidental data loss).	Deleting a parent node does not necessarily delete the child node if it is linked to another parent node.

Examples with Explanations

Hierarchical Model Example: Consider a College Management system:

- **Root Node:** COLLEGE
  - **Child Nodes:** DEPARTMENT and ADMINISTRATION
  - Under DEPARTMENT, we have child nodes COURSE and FACULTY.
- *Limitation:* If a COURSE is co-taught by two separate departments, the Hierarchical Model cannot easily map this, because a child node cannot point to two parent departments. It would require duplicating the course record.

Network Model Example:

Consider a Product Orders system:

- **Parent Nodes:** STORE_A and STORE_B
- **Child Node:** PRODUCT_X
- *Explanation:* PRODUCT_X can be stocked and supplied by both STORE_A and STORE_B simultaneously. In the Network Model, the PRODUCT_X record can maintain direct, physical pointer relationships to both parent nodes without data duplication.

4. Explain about integrity constraints over relations.

Integrity constraints are rules enforced by DBMS to ensure accuracy, consistency, and validity of data in relations. They are defined during schema creation and checked on INSERT, UPDATE, DELETE operations.The four primary relational integrity constraints are:

1. Domain Constraints

- It specifies that the value inside every individual attribute cell must be an **atomic (indivisible) value** belonging to the defined domain of that column.
- It restricts data types, value ranges, and formats.
- **Example:** If the attribute Age is defined with a domain configuration of INTEGER and a check condition Age >= 18, any attempt to insert a string like "Twenty" or a value like 15 will be rejected by the RDBMS engine.

2. Key Constraints

- It states that an entity set must have a minimal set of attributes that uniquely distinguishes each tuple in a relation. No two distinct rows in a relation can have identical values for all attributes within the key.
- **Example:** In a STUDENT relation, if Roll_No is declared as the key constraint, inserting a new student record with a duplicate Roll_No will fail.

3. Entity Integrity Constraint

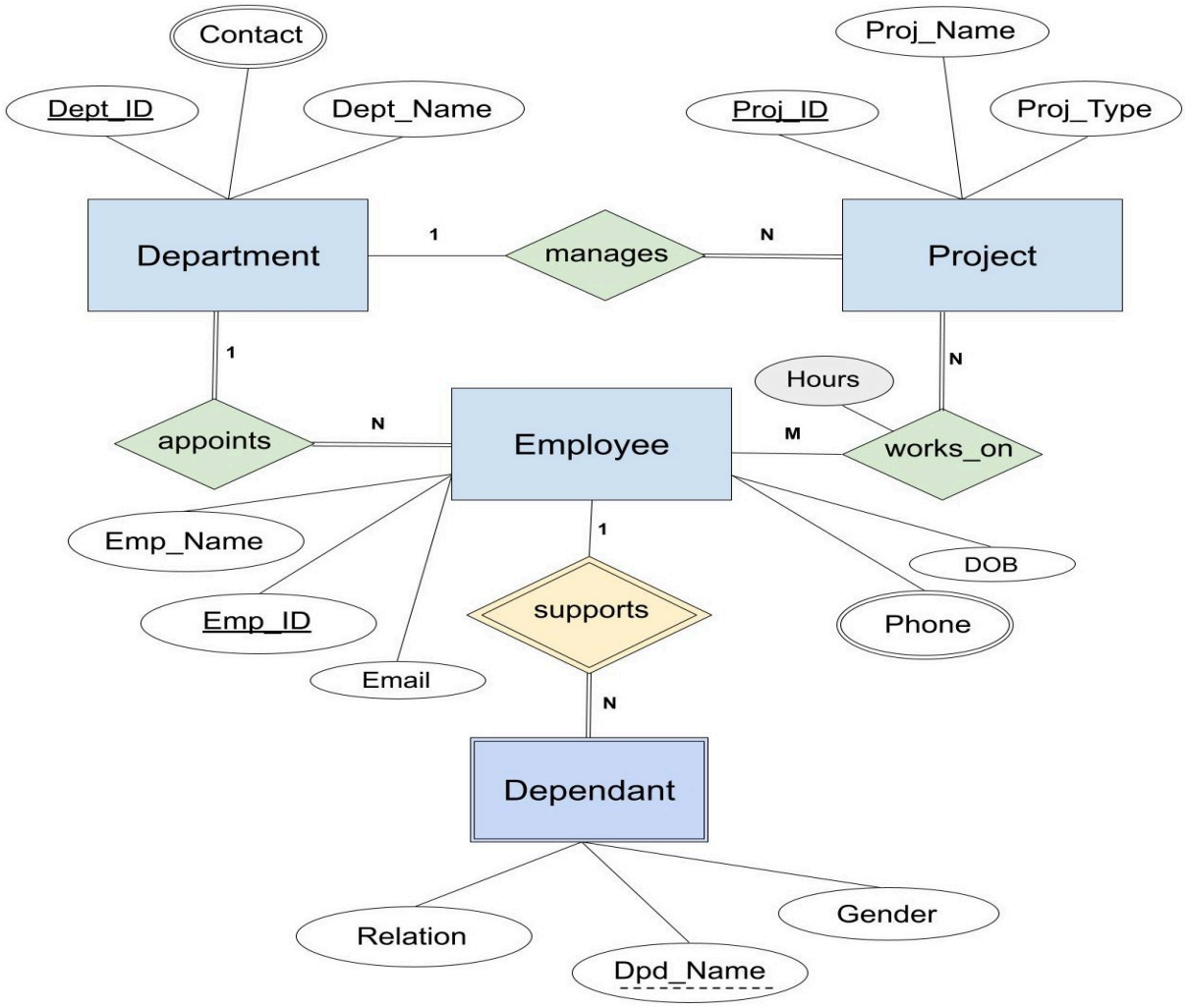
- This rule specifies that **no attribute participating in the primary key of a relation can accept a NULL value**.The primary key is used to identify individual rows uniquely. If a primary key value is allowed to be NULL.
- **Example:** In an EMPLOYEE table where Emp_ID is the primary key, you cannot insert a record with the Emp_ID field left blank or explicitly set to NULL.

4. Referential Integrity Constraint

- It governs relationships between two tables using foreign keys. It states that a tuple in one relation that references another relation must refer to an **existing, valid tuple** within that target relation.
- **Example:** If table ORDERS has a foreign key Customer_ID pointing to CUSTOMER(Customer_ID), you cannot place an order for a Customer_ID that does not exist in the CUSTOMER master table.

5. Draw the ER diagram for a company needs to store information about employees(Identified by ssn, with salary and phone as attributes), departments(Identified by dno, with dname and budget as attributes), and children of employees(With name and age as attributes). Employees work in departments, each department is managed by an employee, a child must be identified uniquely by name when the parent(Who is an employee; assume that only one parent works for the company) is known. We are not interested in information about a child once the parent leaves the company.

6. Create ER diagram for each of the followings: Each company operates four departments and each department belongs to one company. Each department in part (a) employes one or more employees and each employee works for one department. Each of the employees in part(b) may or may not have one or more dependents.



MODULE 3(5 marks)

1. Explain Data Definition Language (DDL) with suitable commands.

- **Definition**: DDL provides commands to define, modify, and manage the physical structure or schema of database objects like tables, views, schemas, and indexes.
- **Key Commands**:
  - CREATE: Initializes a new database object.
  - SQL>CREATE TABLE Employees (Emp_ID INT, Name VARCHAR(50));
  - ALTER: Modifies the column layout or configuration of an existing table.
  - SQL>ALTER TABLE Employees ADD Salary INT;

- DROP: Completely removes a structural object and its underlying data from the database.
- SQL>DROP TABLE Employees;
- TRUNCATE: Clears all row tuples from a targeted table while preserving the empty structural schema framework.
- SQL>TRUNCATE TABLE Employees;

2. Write suitable SQL query to display minimum, maximum, average, and total salaries earned by employees.

```
SQL>SELECT
        MIN(SAL) AS Minimum_Salary,
        MAX(SAL) AS Maximum_Salary,
        AVG(SAL) AS Average_Salary,
        SUM(SAL) AS Total_Salary
    FROM EMPLOYEES;
```

Explanation

- `MIN(SAL)` → Displays minimum salary
- `MAX(SAL)` → Displays maximum salary
- `AVG(SAL)` → Displays average salary
- `SUM(SAL)` → Displays total salary of all employees
- `SAL` is the salary column of the `EMPLOYEES` table.

3. Explain Data Manipulation Language (DML) with suitable commands.

- **Definition**: DML encompasses commands that access, modify, insert, or clean up actual data values within the structural frameworks configured by DDL.
- **Key Commands**:
  - **SELECT:** Retrieves records from one or more tables.
  - SQL>SELECT * FROM Employees WHERE Salary > 50000;
  - **INSERT:** Populates new data rows into a specific table.
  - SQL>INSERT INTO Employees (Emp_ID, Name, Salary) VALUES (1, 'Koulik', 60000);
  - **UPDATE:** Modifies existing attributes or data fields matching specified filters.
  - SQL>UPDATE Employees SET Salary = 65000 WHERE Emp_ID = 1;
  - **DELETE:** Purges selected data rows from a relation based on conditions.
  - SQL>DELETE FROM Employees WHERE Emp_ID = 1;

4. Explain projection operation in relational algebra with suitable example.

- **Definition**: The projection operation (Π) isolates specific attributes (vertical columns) from a relation while filtering out unlisted columns and automatically removing duplicate rows from the final result set.
- **Mathematical Notation**: \Pi_{A_1, A_2, ..., A_n}(R)$ where A_n represent the target attributes and R is the relation.
- **Example**: Consider a relation Student(Roll_No, Name, Department). To list only the names and departments of all students, the expression is:
  \Pi_{Name, Department}(Student)

5. State the difference between cartesian product and natural join in context of relational algebra.

- **Cartesian Product ($\times$)**:
  - Combines every single tuple of the first relation with every single tuple of the second relation unconditionally.
  - The resulting degree (number of columns) is the sum of the degrees of both relations.
  - Does not require the participating tables to share common attributes.
- **Natural Join (⋈)**:

- Combines tuples from two relations based strictly on matching values across columns that share identical names.
- Automatically omits duplicate common attribute columns from the final schema layout.
- Requires at least one common attribute with matching data domains between the relations.

6. Give an example of how to use the rename operation in relational algebra.

- **Definition**: The rename operation (ρ) alters the identifier name of a relation or its specific internal attributes, preventing naming conflicts during self-joins or intermediate data assignments.
- **Syntax: ρ _New_Relation_Name(Old_Relation_Name)**
- **Example**: To evaluate a self-join or rename an existing table named Employee to an alias identifier Staff, the operation is expressed as:
  ρ_{Staff}(Employee)
- To rename both the relation name and its corresponding attributes, use:
  ρ_{Staff(ID, Full_Name)}(π _{Emp_ID, Name}(Employee))

7. Explain the following i. Data Definition Language. ii. Data control Language. iii. Views

- **i. Data Definition Language (DDL)**: Used to define, alter, and delete the structural schema configurations of database files and entities without altering data contents directly. Key commands include CREATE, ALTER, and DROP.
- **ii. Data Control Language (DCL):** Enforces access permissions, user authorizations, and transactional security profiles over database objects. Key commands include GRANT (provisions permissions) and REVOKE (removes permissions).
- **iii. Views:** Virtual database tables generated dynamically from the execution result of a saved SQL lookup query mapping over physical master tables. They do not store local copies of data but provide abstraction and query security.

8. What is a view? How can it be created? Explain with an example.

- **Definition**: A view is a logical, virtual table defined by an SQL query that surfaces a specialized segment of data from one or more underlying physical tables.
- **Creation Syntax**:
- SQL>CREATE VIEW View_Name AS SELECT Columns FROM Table_Name WHERE Condition;
- **Example**: To build a view named CSE_Faculty targeting professor profiles belonging to the computer science department from a physical master table named Instructors:
- SQL>CREATE VIEW CSE_Faculty AS SELECT Teacher_ID, Name, Specialization FROM Instructors WHERE Department = 'CSE';
- This virtual object can then be queried directly like a standard database table:
- SQL>SELECT * FROM CSE_Faculty;

9. What is a join operator?

Explain about conditional join and natural join with syntax and example.

- **Join Operator**: A binary relational operator that merges related row tuples from multiple tables into composite rows based on a designated evaluation rule.
- **Conditional Join (Theta Join, $\bowtie_\theta$)**: Links records based on an explicit condition matching criteria θ utilizing typical logical comparison operators (**=, ≠ , <, >, ≤ , ≥**).
  - *Syntax*: R⋈_{Condition} S
  - *Example*: Employees⋈ _{Employees.Dept_ID = Departments.ID} Departments
- **Natural Join (⋈)**: Joins records across two tables implicitly by matching equivalent values within columns sharing identical names, dropping duplicate identifier columns automatically.

    - *Syntax*: R⋈ S
    - *Example*: Students⋈ Enrollments (where both relations contain a matching Roll_No column).

10. How can we compare using null values?

Explain about logical connectives with examples.

- **Comparing Null Values**: NULL represents missing, unknown, or unassigned data states. It cannot be compared using standard operators like = or != because any comparison operation against a NULL value resolves to UNKNOWN. Instead, the specialized unary operators IS NULL and IS NOT NULL must be utilized.
    - *Example*: SELECT * FROM Employees WHERE Manager_ID IS NULL;
- **Logical Connectives**: SQL utilizes three-valued logic (TRUE, FALSE, UNKNOWN) driven by conditional connectives to filter queries:
    - AND: Requires both individual conditional expressions to resolve to true.
        - *Example*: WHERE Department = 'CSE' AND Salary >= 50000
    - OR: Evaluates to true if at least one conditional expression resolves to true.
        - *Example*: WHERE Department = 'CSE' OR Department = 'ECE'
    - NOT: Inverts the logical evaluation outcome of a targeted condition.
        - *Example*: WHERE NOT (Department = 'Civil')

10 MARKS

1. Discuss different types of outer join with example.

Outer Join is a powerful relational join operation that returns not only the matching rows from two tables (like an Inner Join) but also the non-matching rows from one or both of the participating tables. When there is no match in the joined table, the database fills the missing values with NULL.

Why Outer Joins are Used

Standard joins filter out data that does not satisfy the join condition. Outer joins are essential for reporting where we need to maintain "entity integrity," such as identifying customers who have not yet placed an order or departments that have no employees currently assigned.

Types of Outer Joins

- **Left Outer Join ($⟕$):**
  This operation returns all rows from the "left" table and the matching rows from the "right" table. If a row in the left table does not have a corresponding match in the right table, the resulting row contains NULL values for all columns of the right table.
    - *Example:* If we join EMPLOYEE (Left) with DEPARTMENT (Right), a Left Join will list all employees, even those not assigned to a department.
- **Right Outer Join ($⟖$):**
  This is the inverse of the Left Outer Join. It returns all rows from the "right" table and the matching rows from the left table. If there is no match for a row in the right table, the left table's columns appear as NULL.
    - *Example:* In the same scenario, a Right Join would list all departments, even if those departments currently have zero employees.
- **Full Outer Join ($⟗$):**
  This operation combines the functionality of both Left and Right Outer Joins. It returns all rows from both tables. When a row in one table does not have a match in the other, NULL values are used to fill the missing data from the non-matching side.
    - *Example:* This provides a complete view of both entities, showing unassigned employees and empty departments simultaneously.

2. Discuss different types of integrity constraint with example.

Integrity constraints are rules established on database schemas to guarantee data accuracy, consistency, and reliability. They prevent the entry of invalid data into the database.

1) Domain Constraints

- **Definition:** These constraints define the valid set of values (domain) that an attribute can hold. This includes the data type (integer, string, date), size, and explicit rules (like CHECK constraints).

Example: An `Age` column must be an `INTEGER` and must be greater than 18.
CREATE TABLE Voter (Voter_ID INT, Age INT CHECK (Age >= 18));

2) Entity Integrity Constraint

- **Definition:** This rule states that no attribute participating in the **Primary Key** of a relation can accept a `NULL` value.
- **Reasoning:** Since the primary key uniquely identifies each row, a `NULL` value would mean an unidentified record, defeating the purpose of the key.
- **Example:** In a `STUDENT` table, the `Roll_No` acts as the primary key. If you try to insert a student without a `Roll_No`, the database engine throws an error.

3) Referential Integrity Constraint

- **Definition:** This constraint governs the relationship between two tables. It states that if a **Foreign Key** exists in a child table, its value must perfectly match a valid Primary Key value in the parent table (or be strictly `NULL`).
- **Example:** An `EMPLOYEE` table has a `Dept_ID` (Foreign Key) referencing the `DEPARTMENT` table (Primary Key). You cannot assign an employee to `Dept_ID = 99` if department 99 does not exist.

4) Key Constraints (Unique Constraints)

- **Definition:** Ensures that certain columns (or combinations of columns) that are **Candidate Keys** remain perfectly unique across all tuples in the relation, even if they aren't the designated Primary Key.

Example: In a `USER` table, while `User_ID` is the Primary Key, the `Email_Address` must also be unique across all rows.
CREATE TABLE Users (User_ID INT PRIMARY KEY, Email VARCHAR(100) UNIQUE);

3. i) Write suitable relational algebra operations to display columns Regno, Section from the table Students where branch is EE.ii) Write suitable relational algebra operation to select tuples from the table Book where the topic is “Database” and author is “Ram”.

1. Introduction to Relational Algebra

Relational algebra is a procedural query language that takes one or two relations as input and produces a new relation as output.

Part (i): Displaying specific columns based on a condition

Goal: To retrieve Regno and Section for all students whose branch is 'EE' from the Students table.

- **Step 1 (Selection):** We first filter the Students table to isolate only the tuples where the branch is 'EE'.
  - Expression: $\sigma_{branch='EE'}(Students)$
- **Step 2 (Projection):** From the resulting filtered set, we extract only the required columns: Regno and Section.
  - Expression: $\Pi_{Regno, Section}(\sigma_{branch='EE'}(Students))$

Final Expression: $\Pi_{Regno, Section}(\sigma_{branch='EE'}(Students))$

- **Explanation:** The engine first scans the Students table, discards every row that does not have "EE" in the branch column, and then takes the remaining rows and removes all columns except Regno and Section.

Part (ii): Selecting tuples based on multiple conditions

Goal: To retrieve all attributes (tuples) from the Book table where the topic is 'Database' and the author is 'Ram'.

- **Operator Used:** We use the Selection (σ) operator. Since there are two conditions that must **both** be true, we use the logical conjunction operator (AND, denoted as $\wedge$).
- **Expression:** $\sigma_{topic='Database' \wedge author='Ram'}(Book)$

Final Expression:$\sigma_{topic='Database' \wedge author='Ram'}(Book)$

- **Explanation:**
    1. **Predicate (σ):** The condition **topic='Database'∧author='Ram'** acts as a filter.
    2. **Tuple Selection:** Because no projection is performed, the result is the set of all full records (all attributes) that satisfy the specific requirements for topic and author.

4. What are aggregate functions? Explain five built-in aggregate functions.

Aggregate functions are built-in mathematical SQL functions that perform calculations on a set of values (a whole column or grouped subsets of a column) and return a **single, consolidated scalar value**. They are extensively used in data summarization, reporting, and conjunction with the `GROUP BY` clause. *(Note: Aggregate functions ignore `NULL` values, except for `COUNT(*)`).*

Five Built-in Aggregate Functions:

1. **COUNT()**:
    - **Explanation:** Counts the total number of rows or non-null values in a specified column. `COUNT(*)` counts all rows including those with nulls.

Example: Find the total number of employees.
SELECT COUNT(*) FROM Employee;

2. **SUM()**:
    - **Explanation:** Calculates the total arithmetic sum of all non-null numeric values in a column.

Example: Calculate the total monthly payroll.
SELECT SUM(Salary) FROM Employee;

3. **AVG()**:
    - **Explanation:** Calculates the arithmetic mean (average) of a set of numeric values.

Example: Find the average marks of a class.
SELECT AVG(Marks) FROM Student;

4. **MAX()**:
    - **Explanation:** Identifies and returns the highest (maximum) value from a specified column. It works on numeric, string, and date data types.

Example: Find the most expensive product.
SELECT MAX(Price) FROM Product;

5.MIN():
    - **Explanation:** Identifies and returns the lowest (minimum) value from a specified column.

Example: Find the date of the oldest order.
SELECT MIN(Order_Date) FROM Orders;

5. Explain the structure of basic form of an SQL query. What is cursor in database? How can you create an user defined cursor to store all manager details in SQL?

The fundamental structure of an SQL query follows a specific logical order of execution to retrieve and manipulate data.
SQL>SELECT [DISTINCT] column_list
FROM table_list
[WHERE condition]
[GROUP BY column_list]

[HAVING group_condition]
[ORDER BY column_list [ASC|DESC]];

Database Cursor

SQL is inherently **set-based**, meaning it operates on entire tables or result sets simultaneously. However, many applications require **row-by-row** processing (e.g., performing complex calculations on each record).

A **Cursor** is a database control structure that serves as a pointer to a specific row within a result set. It allows the program to iterate through the result set one row at a time, fetch data, and process it procedurally.

Creating a User-Defined Cursor for Manager Details

To store and process all records where the designation is 'Manager', one must declare, open, fetch, and close the cursor.

Example in PL/SQL:

```
SQL>DECLARE
   -- 1. Declare variables to store data
   v_id   Employees.EmpID%TYPE;
   v_name Employees.Name%TYPE;

   -- 2. Define the cursor
   CURSOR mgr_cursor IS
      SELECT EmpID, Name
      FROM Employees
      WHERE Designation = 'Manager';
BEGIN
   -- 3. Open the cursor
   OPEN mgr_cursor;

   -- 4. Iterate through rows
   LOOP
      FETCH mgr_cursor INTO v_id, v_name;
      EXIT WHEN mgr_cursor%NOTFOUND; -- Exit loop when no rows left

      -- Process the fetched record
      DBMS_OUTPUT.PUT_LINE('Manager ID: ' || v_id || ' | Name: ' || v_name);
   END LOOP;

   -- 5. Close the cursor
   CLOSE mgr_cursor;
END;
```

6. Consider the given relations: STUDENT(SID, Name, Dept, Age) COURSE(CID, CName, Dept) , ENROLL(SID, CID, Marks) Write Relational Algebra expression for the following queries:

• Find names of all students.

We use the **Projection (π)** operator to extract only the name attribute.

- **Expression: π _{Name}(STUDENT)**

• Find names of students whose age is greater than 20.

We use the **Selection (σ)** operator to filter rows, followed by **Projection** to isolate the names.

- **Expression: π _{Name}(σ _{Age > 20}(STUDENT))**

• Find students enrolled in DBMS.
We perform a **Natural Join ($\bowtie$)** across the three tables to associate students with their courses, filter by the course name, and then project the student names.

- **Expression: π _{Name}(STUDENT ⋈ ENROLL ⋈ σ _{CName='DBMS'}(COURSE))**

• Find students who are not enrolled in any course.
We find all student IDs, subtract the set of IDs found in the ENROLL table, and join the result back with STUDENT to get the names.

- **Expression:π _{Name}(STUDENT ⋈ (π_{SID}(STUDENT) - π _{SID}(ENROLL)))**

• Find students who are enrolled in all courses.
This requires the **Division (/)** operator. We divide the set of all enrolled pairs by the set of all available course IDs.

- **Expression: π _{Name}(STUDENT ⋈ (π _{SID, CID}(ENROLL) ÷ π _{CID}(COURSE)))**

7. What are the differences between natural join and cross product of two relations? Explain with example. Discuss each of followings- (a)Natural Join b) equi-join c) self join d) inner join e)outer join

Basis of Difference	Cross Product (Cartesian Product)	Natural Join
Definition	Combines **every tuple** of first relation with **every tuple** of second relation.	Combines only those tuples where **common attributes** have equal values.
Condition	No condition is applied.	Equality condition on all common attributes.
Result Size	Very large (**m × n** tuples)	Much smaller (only matching tuples).
Duplicate Columns	All columns from both relations are retained (including duplicates).	Common columns appear **only once**.
Purpose	Rarely used directly (mostly intermediate result).	Used to combine related data meaningfully.
Symbol	R × S	R ⋈ S

Example:

Let Employee (2 tuples) and Department (3 tuples) have one common attribute dno.

- Cartesian Product: Produces 2 × 3 = 6 tuples.
- Natural Join: Produces only those tuples where dno matches (e.g., 3 or 4 tuples).

Discussion of Join Types

- **(a) Natural Join (⋈):** Links relations based on all columns with identical names. It is the most common form of "equi-join" where the join condition is implicit.

- **(b) Equi-Join:** A join that uses the equality operator (=) to link relations based on specific attributes. Unlike a Natural Join, it keeps both copies of the join attribute.
- **(c) Self Join:** A table is joined with itself. This is useful for hierarchical data, such as finding an employee and their manager within the same EMPLOYEE table.
- **(d) Inner Join:** The standard join that returns only those tuples that satisfy the join condition. If a tuple has no match in the other table, it is excluded from the result.
- **(e) Outer Join:** An operation that preserves tuples that would otherwise be lost. It includes unmatched rows from one or both tables, filling the missing attribute values with NULL.
    - *Left Outer Join:* Keeps all from the left table.
    - *Right Outer Join:* Keeps all from the right table.
    - *Full Outer Join:* Keeps all rows from both tables.

MODULE 4

1. Explain Functional dependency with suitable example.

A Functional Dependency (FD) is a constraint that determines the relationship between two sets of attributes in a relation. If an attribute Y is functionally dependent on an attribute X, it is denoted as X $\rightarrow$ Y. This means that for any two tuples in the relation, if their X values are the same, their Y values must also be the same.

- **Determinant:** The left side of the arrow (X).
- **Dependent:** The right side of the arrow (Y).
- **Example:** In an Employee table with attributes (Emp_ID, Name, Salary), the Emp_ID uniquely identifies the Name and Salary.
    - Therefore, Emp_ID $\rightarrow$ Name and Emp_ID $\rightarrow$ Salary.

2. Explain transitive dependency with suitable example.

A transitive dependency occurs when a non-prime attribute functionally determines another non-prime attribute. Formally, if X $\rightarrow$ Y and Y $\rightarrow$ Z exist in a relation (and Y is not a candidate key), then X $\rightarrow$ Z is a transitive dependency. Transitive dependencies violate the rules of Third Normal Form (3NF).

- **Example:** Consider a Book table with (Book_ID, Author_ID, Author_Nationality).
    - Book_ID $\rightarrow$ Author_ID (A book has a specific author).
    - Author_ID $\rightarrow$ Author_Nationality (An author has a specific nationality).
    - Therefore, Book_ID $\rightarrow$ Author_Nationality is a transitive dependency, leading to potential data anomalies.

3. Define Third Normal Form (3NF) with example.

A relation is in Third Normal Form (3NF) if it is already in Second Normal Form (2NF) and contains no transitive partial dependencies. For every non-trivial functional dependency X $\rightarrow$ Y, one of the following conditions must hold true:

1. X is a Super Key.
2. Y is a Prime Attribute (an attribute that is part of a candidate key).

- **Example:** A Student table (Roll_No, Name, Dept_ID, Dept_Name) where Roll_No $\rightarrow$ Dept_ID and Dept_ID $\rightarrow$ Dept_Name. This is not in 3NF due to the transitive dependency. To convert it to 3NF, decompose it into two tables: Student(Roll_No, Name, Dept_ID) and Department(Dept_ID, Dept_Name).

4. What is redundancy? Explain with suitable example. What are the anomalies created by data redundancy? Explain.

Redundancy is the unnecessary repetition of the same piece of data across multiple records in a database.

- **Example:** Storing the Department_Name "Computer Science" multiple times for every student enrolled in that department in a single Student table.

Anomalies Created:

- **Insertion Anomaly:** Inability to add data to the database due to the absence of other data. (e.g., Cannot add a new department until a student enrolls in it).
- **Deletion Anomaly:** Accidental loss of related data when deleting a record. (e.g., Deleting the only student in a department also deletes the department's information).
- **Updation Anomaly:** Data inconsistency when a redundant value is updated in one place but missed in another. (e.g., Changing the HOD's name requires updating multiple student records).

5. Explain the classification of functional dependency.

Functional dependencies are primarily classified into three types based on the relationship between the determinant and dependent attributes:

- **Trivial Functional Dependency:** Occurs when the right-hand side is a subset of the left-hand side. $X \rightarrow Y$ is trivial if $Y \subseteq X$. (e.g., $A, B \rightarrow A$).
- **Non-Trivial Functional Dependency:** Occurs when the right-hand side is not a subset of the left-hand side. $X \rightarrow Y$ is non-trivial if $Y \not\subseteq X$. (e.g., $A, B \rightarrow C$).
- **Completely Non-Trivial Functional Dependency:** A stricter form where the determinant and dependent sets share no common attributes. $X \rightarrow Y$ is completely non-trivial if $\mathbf{X \cap Y = \emptyset}$.

6. Consider the relation schema R(A,B,C,D,E,F) and the functional dependencies A->B, C->DF, AC->E, D->F. Find the candidate keys of this relation R.

To find the candidate key, we must find an attribute closure that encompasses all attributes {A, B, C, D, E, F}. Attributes A and C do not appear on the right side of any functional dependency, so they must be part of the candidate key. Let's calculate the closure of (AC)^+:

- Start: (AC)^+ = {A, C}
- Using $A \rightarrow B$: (AC)^+ = {A, B, C}
- Using $C \rightarrow DF$: (AC)^+ = {A, B, C, D, F}
- Using $AC \rightarrow E$: (AC)^+ = {A, B, C, D, E, F}
  Since (AC)^+ derives all the attributes in the relation, **AC** is the sole candidate key for relation R.

7. Define and explain 4NF with suitable example.

A relation is in Fourth Normal Form (4NF) if it is in Boyce-Codd Normal Form (BCNF) and contains no non-trivial Multivalued Dependencies (MVD). An MVD (X \twoheadrightarrow Y$) exists when an attribute determines multiple values of another attribute independently of a third.

- **Example:** Course(Course_Name, Instructor, Textbook). Suppose a course ("DBMS") has multiple independent instructors ("Alice", "Bob") and multiple recommended textbooks ("Book1", "Book2"). Representing this in one table causes severe redundancy because Instructors and Textbooks are independent of each other. To satisfy 4NF, it is decomposed into two tables: Course_Instructor(Course_Name, Instructor) and Course_Textbook(Course_Name, Textbook).

8. Explain join dependencies with fifth normal form with an example.

Fifth Normal Form (5NF), or Project-Join Normal Form (PJNF), ensures that a table cannot be decomposed into smaller tables and then losslessly rejoined. It deals with **Join Dependency (JD)**. A table is in 5NF if every join dependency is implied by the candidate keys.

- **Example:** Consider a ternary relationship Supplier_Part_Project(Supplier, Part, Project). If a supplier supplies a part, a project requires a part, and a supplier supplies a project, we might deduce they supply *that* part for *that* project. If the data strictly adheres to this, leaving it in one table causes redundancy. 5NF dictates decomposing it into three tables: (Supplier, Part), (Part, Project), and (Supplier, Project) to avoid update anomalies while ensuring joining them perfectly reconstructs the original table.

9. Consider the relation R(v,w,x,y,z) with functional dependencies { z->y, y->z, x->y, x->v, vw->x }. Find the closure for {xy} given functional dependencies.

We need to find (xy)^+.

- Step 1: Start with the given set. (xy)^+ = {x, y}
- Step 2: Use y → z. Add z. (xy)^+ = {x, y, z}
- Step 3: Use x → v. Add v. (xy)^+ = {v, x, y, z}
- Step 4: Check other FDs. z → y gives nothing new. vw → x cannot be used because w is not in our closure set.
  Thus, the final closure for {xy} is **{v, x, y, z}**.

10. Given below are two sets of FD's for a relation R(A,B,C,D,E). Are they equivalent? F={A → C, AC → D, E → AD, E → H} and G={A → CD, E → AH}

Two sets of FDs are equivalent if F covers G and G covers F.

- **Check if F covers G:**
  - Find (A)^+ under F: A → C gives {A, C}. AC → D gives {A, C, D}. This covers A → CD in G.
  - Find (E)^+ under F: E → AD gives {A, D, E}. E → H gives {A, D, E, H}. A → C gives {A, C, D, E, H}. This covers E → AH in G.
- **Check if G covers F:**
  - Find (A)^+ under G: A → CD gives {A, C, D}. Covers A → C in F.
  - Find (AC)^+ under G: gives {A, C, D}. Covers AC → D in F.
  - Find (E)^+ under G: E → AH gives {A, E, H}. A → CD gives {A, C, D, E, H}. Covers E → AD and E → H in F.
    Result: Yes, the two sets of functional dependencies F and G are equivalent.

11. "All BCNF relations are in 3 NF but not vice versa". Prove the statement with example.

Proof: The 3NF condition states that for every non-trivial X → Y, either X is a superkey OR Y is a prime attribute. BCNF strictly demands that for every non-trivial X → Y, X MUST be a superkey. Because BCNF is a stricter subset of the 3NF rules, any relation satisfying BCNF automatically satisfies 3NF.

Counterexample (3NF but not BCNF):

Consider a relation R(Student, Course, Instructor) where a student can take multiple courses, a course can have multiple instructors, but an instructor teaches only one course.

- FDs: (Student, Course) → Instructor, and Instructor → Course.
- Candidate Keys: (Student, Course) and (Student, Instructor).
- Check Instructor → Course: Course is a prime attribute, so the relation is in 3NF. However, Instructor is NOT a superkey. Therefore, it violates BCNF. This proves 3NF does not guarantee BCNF.

12. Explain Multivalued dependency with suitable example.

Multivalued Dependency (MVD) occurs when two or more independent multivalued attributes exist in the same table. It represents a situation where the presence of one attribute necessitates the presence of multiple values for another attribute, completely independent of a third attribute. It is denoted by X ↠ Y.

- **Conditions for MVD:**
  1. For a dependency A → B, if for a single value of A, multiple values of B exist, it is A ↠ B.
  2. The relation must have at least three attributes.
  3. The dependent attributes must be independent of each other.
- **Example:** Consider an Employee table with attributes (Emp_ID, Skill, Hobby). An employee can have multiple skills (Java, Python) and multiple hobbies (Reading, Swimming). Skills and hobbies are entirely independent. Thus, Emp_ID ↠ Skill and Emp_ID ↠ Hobby are multivalued dependencies.

10 MARKS

1. Consider a schema R(A,B,C,D) and functional dependencies A->B and C->D. Check that the decomposition of R into R1(AB) and R2(CD) gives lossless join and dependency preserving decomposition or not.

Given: * Schema R(A, B, C, D)
Functional Dependencies (FDs): A → B, C → D. Decomposition: R_1(A, B) and R_2(C, D)

1. Lossless Join Check:

For a decomposition into R_1 and R_2 to be lossless, their intersection must determine either all attributes of R_1 or all attributes of R_2.

- R_1 ∪ R_2 = {A, B} ∪ {C, D} = {A, B, C, D} (Covers all attributes)
- R_1 ∩ R_2 = {A, B} ∩ {C, D} = ∅
  Because the intersection is empty, it cannot determine either R_1 or R_2.
  Conclusion: The decomposition is **Lossy**.

2. Dependency Preserving Check:

A decomposition is dependency preserving if the union of FDs valid on the individual decomposed relations equals the original set of FDs (F).

- FDs for R_1: A → B is valid in R_1. Let's call this F_1.
- FDs for R_2: C → D is valid in R_2. Let's call this F_2.
- F_1 ∪ F_2 = {A → B, C → D}
  Since (F_1 ∪ F_2)^+ is identical to the original F^+, no dependencies are lost.
  Conclusion: The decomposition is **Dependency Preserving**.

2. Consider a relation R(A,B,C,D,E,F,G) with the functional dependencies AB->CD AF->D DE->F C->G F->E G->A Find closure of attribute sets {BG} and {AF}. Illustrate fully functional dependency with example.

Given: Relation R(A, B, C, D, E, F, G) . FDs: AB → CD, AF → D, DE → F, C→ G, F→E, G→ A

1. Closure of {B, G}^+:

- **Step 1:** Start with the given set: {B, G}
- **Step 2:** Use G → A. Add A. Current set: {A, B, G}
- **Step 3:** Use AB → CD. Add C, D. Current set: {A, B, C, D, G}
- **Step 4:** Use C → G. G is already in the set. No more FDs can be applied.
  Result: {B, G}^+ = **{A, B, C, D, G}**

2. Closure of {A, F}^+:

- **Step 1:** Start with the given set: {A, F}

- **Step 2:** Use F → E. Add E. Current set: {A, E, F}
- **Step 3:** Use AF→ D. Add D. Current set: {A, D, E, F}
- **Step 4:** Use DE → F. F is already in the set. No more FDs can be applied.
  Result: {A, F}^+ = **{A, D, E, F}**

3.Fully Functional Dependency Definition & Example:

An attribute Y is **fully functionally dependent** on a set of attributes X if X → Y exists, but Y is not functionally dependent on any proper subset of X. If it does depend on a subset, it is called a partial dependency.

- **Example:** Consider a relation Enrollment(Student_ID, Course_ID, Grade).
  - The FD is {Student_ID, Course_ID} → Grade.
  - A single Student_ID cannot determine the Grade (since a student takes many courses), and a single Course_ID cannot determine the Grade (since many students take a course).
  - Therefore, Grade is fully functionally dependent on the composite key {Student_ID, Course_ID}.

3. Consider the following relation: Shipping (Shipname, Shiptype, Voyageid, Cargo, Port, Date) (note: Date the date the ship arrives in the given port) with functional dependencies- {ShipName->Shiptype, Voyageid->Shipname, Cargo, Shipname, Date->Voyageid, Port} a) Identify the candidate keys. b) Normalize the relation until it is completely normalized.

Given: Relation: Shipping(Shipname, Shiptype, Voyageid, Cargo, Port, Date)
FDs:Shipname → Shiptype**;** Voyageid → Shipname, Cargo**;** {Shipname, Date} → {Voyageid, Port}

a) Identify Candidate Keys: To find candidate keys, we compute closures for combinations of attributes that don't appear on the right-hand side of any FD (specifically Date).

- **Check {Voyageid, Date}^+:** * {Voyageid, Date} determines Shipname, Cargo (using FD 2).
  - Now we have {Voyageid, Date, Shipname, Cargo}.
  - Shipname → Shiptype (using FD 1).
  - Now we have {Voyageid, Date, Shipname, Cargo, Shiptype}.
  - {Shipname, Date} → Port (using FD 3).
  - Final closure covers all attributes: {Voyageid, Date, Shipname, Cargo, Shiptype, Port}.
- **Check {Shipname, Date}^+:**
  - {Shipname, Date} determines Voyageid, Port (using FD 3).
  - Now we have {Shipname, Date, Voyageid, Port}.
  - Voyageid → Cargo (using FD 2).
  - Shipname → Shiptype (using FD 1).
  - Final closure covers all attributes.
    Result: The candidate keys are **{Voyageid, Date}** and **{Shipname, Date}**.

b) Normalize completely (To BCNF):

- **1NF Check:** Assuming all attributes hold atomic values, it is in 1NF.
- **2NF Check:** We have partial dependencies because subsets of our candidate keys determine non-prime attributes:
  - Shipname → Shiptype (Partial dependency on {Shipname, Date})
  - Voyageid → Shipname, Cargo (Partial dependency on {Voyageid, Date})

- **Decompose to 2NF / 3NF:** Remove partial dependencies into their own tables.
    - R_1(\underline{Shipname}, Shiptype) — *In BCNF*
    - R_2(\underline{Voyageid}, Shipname, Cargo) — *In BCNF*
    - R_3(\underline{Voyageid, Date}, Port) — *In BCNF*

    Result: The completely normalized schema contains three relations:

1. **Ship_Details** (Shipname, Shiptype)
2. **Voyage_Details** (Voyageid, Shipname, Cargo)
3. **Schedule** (Voyageid, Date, Port)

4. Define BCNF. How does it differ from 3 NF? Why is it considered as stronger than 3 NF?

1. Definition of BCNF:A relation schema is in Boyce-Codd Normal Form (BCNF) if, for every one of its non-trivial functional dependencies X → Y, the determinant X is a superkey.

2. Difference from 3NF:

- **3NF Condition:** Allows a functional dependency X → Y if X is a superkey **OR** if Y is a prime attribute (part of a candidate key).
- **BCNF Condition:** Strictly dictates that X **MUST** be a superkey. There are no exceptions for prime attributes.

3. Why it is stronger:

BCNF is considered stronger than 3NF because it eliminates all anomalies based on functional dependencies. While 3NF is usually sufficient, it allows a specific type of redundancy: when overlapping candidate keys exist and a non-key attribute determines a prime attribute. BCNF forbids this, meaning any relation in BCNF is automatically in 3NF, but a relation in 3NF is not necessarily in BCNF.

5. Discuss Armstrong's Axioms.

Armstrong's Axioms are a set of fundamental inference rules used to logically derive all functional dependencies implied by a given set of FDs. They are both **sound** (they will not generate false dependencies) and **complete** (they can generate all true dependencies).

Primary Rules (Core Axioms):

1. **Axiom of Reflexivity:** If Y is a subset of X (Y ⊆ X), then X → Y. (These are trivial dependencies, e.g., A, B → A).
2. **Axiom of Augmentation:** If X → Y holds, and Z is any set of attributes, then XZ → YZ also holds. Adding the same attribute to both sides does not break the dependency.
3. **Axiom of Transitivity:** If X → Y holds, and Y → Z holds, then X → Z logically holds.

Secondary Rules (Derived Axioms):

4. **Union Rule:** If X → Y and X → Z, then X → YZ.
5. **Decomposition Rule:** If X → YZ, then X → Y and X → Z hold independently.
6. **Pseudotransitivity Rule:** If X → Y holds, and WY → Z holds, then WX → Z holds.

6. Consider the following relation schema Works(Pname, Cname, salary) Lives(Pname,Street,City) Located_in (Cname, city) Manager(Pname, Mgrname) Write the SQL queries for the following. i) Find the names of all persons who live in the city Bangalore. ii) Retrieve the names of all person of "Infosys" whose salary is between Rs.50000 iii) Find the names of all persons who lives and work in the same city iv) List the names of the people who work for "Tech

M^”along with the cities they live in. v) Find the average salary of "Infosys" persons

i) Find the names of all persons who live in the city Bangalore.

SQL>SELECT Pname FROM Lives WHERE City = 'Bangalore';

ii) Retrieve the names of all person of "Infosys" whose salary is between Rs.50000

SQL>SELECT Pname FROM Works WHERE Cname = 'Infosys' AND salary >= 50000;

iii) Find the names of all persons who lives and work in the same city

SQL>SELECT W.Pname FROM Works W JOIN Lives L ON W.Pname = L.Pname JOIN Located_in Loc ON W.Cname = Loc.Cname WHERE L.City = Loc.city;

iv) List the names of the people who work for "Tech M" along with the cities they live in.

SQL>SELECT W.Pname, L.City FROM Works W JOIN Lives L ON W.Pname = L.Pname WHERE W.Cname = 'Tech M';

v) Find the average salary of "Infosys" persons

SQL>SELECT AVG(salary) AS Average_SalaryFROM Works WHERE Cname = 'Infosys';

MODULE - V 5 marks

1. Explain the pros and cons of Sequential File Organization.

Sequential File Organization stores records in a sequential manner, usually sorted on a key field (e.g., Employee ID or Roll Number).

Pros (Advantages):

- Simple to implement and understand.
- Very efficient for sequential access and batch processing (e.g., generating payroll reports or processing transactions in order).
- Low storage overhead as no extra space is required for indexes.
- Suitable for files that are rarely modified but frequently read in sequence.

Cons (Disadvantages):

- Inefficient for random access queries as it requires scanning from the beginning.
- Insertion and deletion of records are costly because records need to be shifted.
- Poor performance when frequent updates are required.
- Not suitable for online transaction processing (OLTP) systems.

It is mainly used in applications where data is processed sequentially, such as magnetic tapes.

2. Write the differences between B-Tree and B+ Tree.

Feature	B-Tree	B+ Tree
Data Storage	Keys and records stored in both internal and leaf nodes	Only keys in internal nodes; all records stored only in leaf nodes
Leaf Node Structure	Leaf nodes are not linked	Leaf nodes are linked forming a sequential linked list

Range Query Efficiency	Slower (requires tree traversal)	Faster due to linked leaf nodes
Tree Height	Slightly higher	Usually shorter (higher fanout)
Space Utilization	Lower	Better
Usage	Moderate use	Widely used in database systems

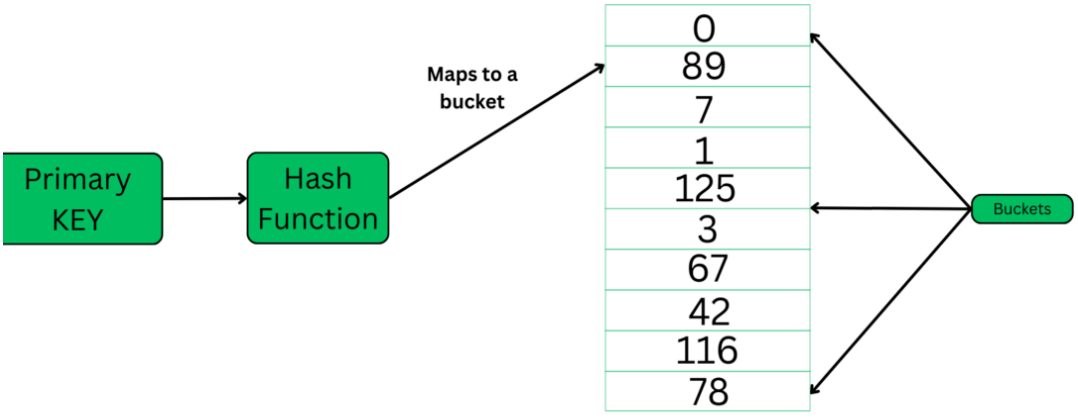
Conclusion: B+ Trees are preferred in DBMS because they support efficient range searches and sequential access.

3. Explain the concept of Hash File Organization with suitable diagram.

Hash File Organization is a direct or random access method in DBMS where a mathematical algorithm—the Hash Function—is applied to a specific column (the Hash Key) to compute the exact physical address where a record is stored.

Key Concepts:

- **Hash Key:** The specific field or attribute (usually the Primary Key) used to determine the location of the record.
- **Hash Function**: Maps the key value to a bucket address.
- **Data Buckets** (Data Blocks): The physical memory locations or disk blocks where the actual records are stored.



Hash Table

4. Define the terms in the context of file and storage management. Seek time, access time, rotational latency.

- **Seek Time**: It is the time taken by the read/write head of the disk to move to the correct track (cylinder) where the required data is located. It is one of the major components of disk access delay.
- **Rotational Latency (Rotational Delay)**: After reaching the correct track, it is the time taken for the disk to rotate so that the desired sector comes under the read/write head. On average, it is half the time for one full rotation of the disk.
- **Access Time**: It is the total time required to locate and read/write the data from the disk. **Access Time = Seek Time + Rotational Latency + Data Transfer Time**

These parameters are critical in evaluating the performance of secondary storage devices.

5. Compare Primary indexing and clustering indexing in file management techniques.

Feature	Primary Indexing	Clustering Indexing
Definition	Index created on the primary key of the file	Index created on a non-primary key field
Uniqueness	Always unique	Can have duplicate values
File Sorting	Data file is physically sorted on primary key	Data file is physically sorted on clustering key
Number per File	Only one primary index per file	Only one clustering index per file
Sparse/Dense	Usually sparse	Can be sparse or dense
Example	Index on Employee ID	Index on Department ID (multiple employees)

Primary index is used with the ordering key, while clustering index improves retrieval on frequently searched non-primary attributes.

6. Disadvantages of B-Tree over B+ Tree.

- **Inefficient Range Queries**: B-Tree does not link its leaf nodes, making range searches and sequential access slower.
- **Higher Space Overhead**: Records are stored in internal nodes as well, leading to duplication and more space consumption.
- **More Disk Accesses**: Due to lower fanout, the height of the tree is usually greater, resulting in more disk I/O operations.
- **Complex Implementation**: Handling data in internal nodes makes deletion and insertion more complicated.
- **Poor for Database Systems**: Most modern DBMS prefer B+ Tree because it supports better search, insertion, and range scan performance.

Due to these limitations, B+ Trees are more widely used in file indexing.

7. Explain how secondary indexing can help in join operation.

Secondary Indexing is an index created on non-primary key attributes (non-ordering attributes). It helps in faster retrieval when the search condition is based on non-key fields.

Role in Join Operation:

- In join operations (especially equi-join), secondary indexes on join attributes (e.g., foreign keys) allow the DBMS to quickly locate matching records without scanning the entire table.
- It significantly improves the performance of nested loop joins and merge joins.
- Reduces the number of block accesses and I/O operations.
- **Example**: When joining Employee and Department tables on DeptID, a secondary index on DeptID in the Employee table helps in quickly finding all employees belonging to a particular department.

Conclusion: Secondary indexes are very useful in large databases to speed up join queries and improve overall query performance.

10 MARKS

1. Explain the concept of Sequential File Organization with suitable diagram.

File Organization refers to the logical and physical arrangement of records within a storage file. In **Sequential File Organization**, records are stored consecutively in a fixed linear sequence. It is primarily classified into two methods:

1. Pile File Method

In this method, records are stored sequentially in the exact chronological order of their arrival (**First-Come, First-Served**). There is no inherent sorting based on keys or attributes.

- **Insertion:** Extremely fast (O(1)). A new record is simply appended to the absolute end of the file (EOF) after the last record , without any searching.
- **Search:** Slow (O(n)). Because the data is completely unsorted, finding a specific record requires a linear scan from the very beginning of the file.

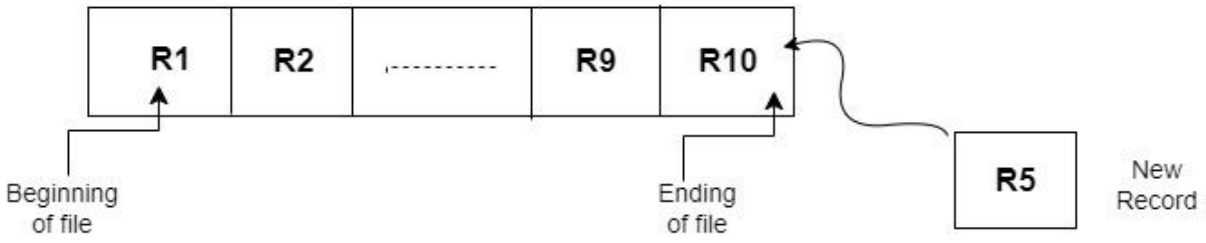


Fig: Insertion of record in pile file method

2. Sorted File Method

In this method, records are strictly maintained in a logical and physical sequence governed by an **Ordering Key** in either ascending or descending order.

- **Insertion:** Slow (O(n)). To insert a new record , the system must first locate the correct position using Binary Search and then physically shift all subsequent records downward to create an empty slot.
- **Search:** Fast (O(log n)). Since the records are already sorted, efficient search algorithms like **Binary Search** can be used to quickly retrieve data.

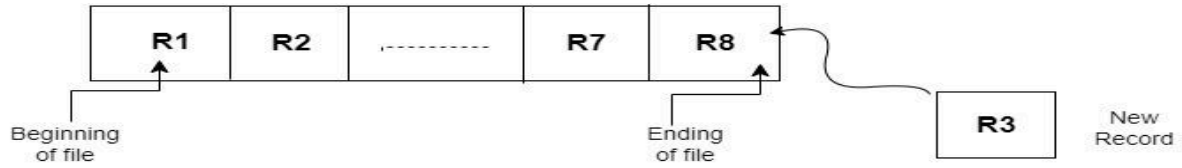


Fig: Insertion of record in sorted file method

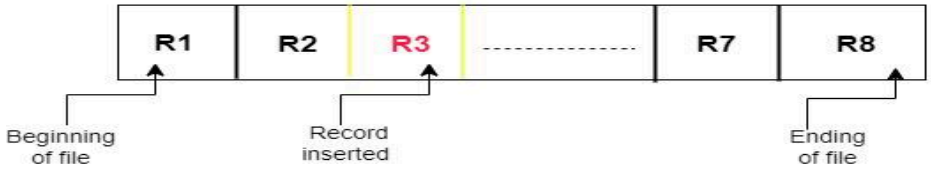


Fig: Record inserted in sorted manner

2. Consider insertion sequence- 2,3,5,11,17,19,23,29,31. Construct B+ Tree for the case where the number of pointers that will fit in one node is 4.

Structural Invariants for Order p = 4:

- **Internal Nodes:** Can hold a maximum of 4 pointers and p - 1 = 3 keys. They must hold at least $\lceil 4/2 \rceil = 2$ pointers.
- **Leaf Nodes:** Can hold up to 3 data values/keys, along with a final pointer chaining to the next sequential leaf node.

Step-by-Step Insertion Tracing:

Given Sequence: 2, 3, 5, 11, 17, 19, 23, 29, 31

- **Step 1: Insert 2, 3, 5**
  The root starts as a leaf node. All three values fit comfortably without violating constraints.

Leaf: [2 | 3 | 5]

- **Step 2: Insert 11**
  The leaf node overflows [2, 3, 5, 11]. It splits down the middle. Keys 2, 3 stay on the left leaf; keys 5, 11 move to the right leaf. The minimum key of the right leaf (5) is copied up to form a new internal root node.

```
   [ 5 ]
   /   \
[2 | 3] -> [5 | 11]
```

- **Step 3: Insert 17**
  Value 17 is guided to the rightmost leaf node since $17 \ge 5$. It fits without overflow.

```
   [ 5 ]
   /   \
[2 | 3] -> [5 | 11 | 17]
```

- **Step 4: Insert 19**
  Value 19 targets the rightmost leaf, creating an overflow condition: [5, 11, 17, 19]. The node splits. Keys 5, 11 stay left; keys 17, 19 form the right leaf. The right side's smallest key (17) is copied up to the root.
- Plaintext

```
   [ 5 | 17 ]
   /   |   \
[2 | 3]->[5 | 11]->[17 | 19]
```

- **Step 5: Insert 23**
  Value 23 goes into the rightmost leaf because 23 ≥ 17. It fits seamlessly.

```
   [ 5 | 17 ]
   /   |   \
[2 | 3]->[5 | 11]->[17 | 19 | 23]
```

- **Step 6: Insert 29**
  Value 29 causes the rightmost leaf to overflow: [17, 19, 23, 29]. It splits. Keys 17, 19 stay left; keys 23, 29 move right. The value 23 copies up into the root node.

```
   [ 5 | 17 | 23 ]
   /   |   |   \
[2 | 3]->[5 | 11]->[17 | 19]->[23 | 29]
```

- **Step 7: Insert 31**
  Value 31 goes into the last leaf node [23, 29]. It expands to [23, 29, 31]. Since it does not exceed 3 keys, no further splitting occurs.

Final B+ Tree Visual Structure:

```
          [ 5  |  17  |  23 ]   <-- Root (Internal Node)
          /    |     |    \
    [2 | 3]  [5 | 11] [17 | 19] [23 | 29 | 31]  <-- Linked Leaves
```

3. Given a block can hold either 3 records or 10 key pointers. A database contains n records, then how many blocks do we need to hold the data file and the dense index.

To determine the overall block requirement, we must evaluate both the data file allocation and the dense index independently.

1. Data File Block Calculation:

- Total number of records in the database = n
- Maximum capacity of a single data block = 3 records
- Since records cannot be split across separate block boundaries, we use the ceiling function (⌈ ⌉) to handle fractional allocations.
  Blocks required for Data File = ⌈n/3⌉

2. Dense Index Block Calculation:

- By definition, a **Dense Index** maintains exactly *one distinct index entry* for every single record present in the primary data file.
- Therefore, the total number of index entries = n.
- Maximum capacity of an index block = 10 key pointers}
  Blocks required for Dense Index = ⌈n/10⌉

3. Total Blocks Formula:

Summing up both portions provides the total physical block space required to manage the relation system:

Total Blocks Required = ⌈n/3⌉ + ⌈n/10⌉

4. The order of a leaf node in a B+ tree is the maximum number of (value, data record pointer) pairs it can hold. Given that the block size is 1K bytes, data record pointer is 7 bytes long, the value field is 9 bytes long and a block pointer is 6 bytes long, what is the order of the leaf node?

1. Given Parameters:

- Total Block Size (B) = 1 KB = 1024 bytes}
- Data Record Pointer size (R_p) = 7 bytes
- Value / Search Key field size (V) = 9 bytes}
- Block Pointer size (P_b) = 6 bytes}
- Let the order of the leaf node be m.

2. Leaf Node Structure Formulation:

According to the problem statement, a leaf node contains m pairs of (value, data record pointer). Additionally, a structural leaf node in a B+ tree requires exactly **one block pointer** at the end of the node to connect linearly to the next sibling leaf node. Therefore, the structural formula bounding the block layout capacity is:

(m × Size of Pair) + Size of Sibling Block Pointer ≤ Block Size

$$(m \times (V + R_p)) + P_b \leq B$$

3. Algebraic Calculation Steps:

Substitute the given numerical metrics into our structural equation:

$(m \times (9 + 7)) + 6 \leq 1024$

$16m + 6 \leq 1024$

$16m \leq 1024 - 6$

$16m \leq 1018$

$m \leq 1018 / 16$

$m \leq 63.625$

4. Final Order Assignment:

Since the structural order parameter m must represent a whole integer, we round down to the nearest integer. m = 63

Conclusion: The maximum order of the leaf node under these block-size design constraints is **63**.

MODULE - VI 5 marks

1. Explain serial schedule with example.

Serial Schedule is a schedule in which transactions are executed one after another (sequentially). No interleaving of operations occurs between different transactions.

Characteristics:

- Transactions run completely one by one.
- Very safe from concurrency issues.
- Produces correct results but low throughput and poor resource utilization.

Example:

Transaction T1: Read(A), Write(A)
Transaction T2: Read(B), Write(B)

Serial Schedule:

T1 → T2
(or T2 → T1)
All operations of T1 finish before T2 starts. This is simple but inefficient for multi-user systems.

2. Explain non-serial schedule with example.

Non-Serial Schedule (also called Concurrent Schedule) is a schedule where operations from different transactions are interleaved.

Characteristics:

- Improves CPU and resource utilization.
- Increases throughput.
- May lead to concurrency problems like lost update, dirty read, etc. if not properly managed.

Example:

T1: R(A), W(A)
T2: R(B), W(B)

Non-Serial Schedule:

T1: R(A) → T2: R(B) → T1: W(A) → T2: W(B)
Here, operations of T1 and T2 are interleaved.

3. Explain Transaction Control Language (TCL) with suitable commands.

Transaction Control Language (TCL) commands are used to manage transactions in the database. They control how changes made by DML statements are handled.

Main TCL Commands:

- **COMMIT**: Permanently saves all changes made in the current transaction. COMMIT;
- **ROLLBACK**: Undoes all changes made in the current transaction. ROLLBACK;
- **SAVEPOINT**: Creates a checkpoint within a transaction to which you can later roll back. SAVEPOINT sp1;
- **SET TRANSACTION**: Sets properties for the current transaction.

Example:SQL>UPDATE Employee SET salary = salary + 5000;

```
SAVEPOINT sp1;
UPDATE Employee SET salary = salary + 10000;
ROLLBACK TO sp1;  -- Rolls back only the second update
COMMIT;
```

4. Define Lock. Discuss Two-Phase Locking protocol with suitable diagram.

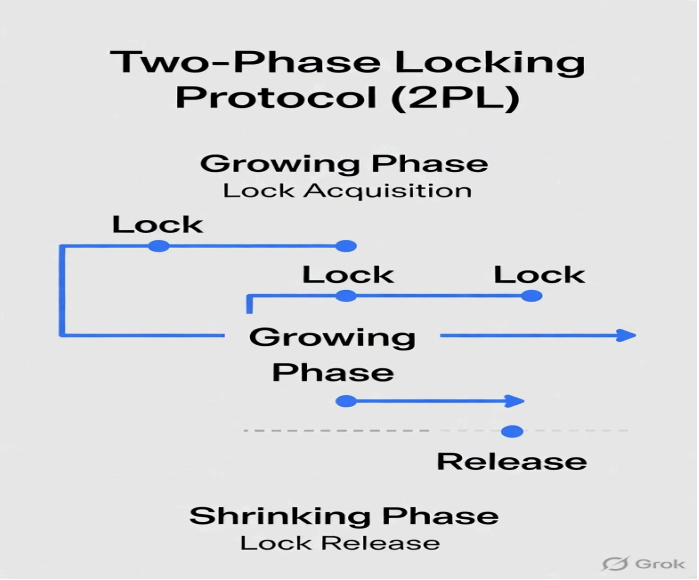
Lock is a mechanism used to control concurrent access to data items. It prevents multiple transactions from accessing the same data simultaneously in conflicting ways.

Two-Phase Locking (2PL) Protocol:

It consists of two phases:

1. **Growing Phase**: Transaction can acquire locks (shared or exclusive) but cannot release any lock.
2. **Shrinking Phase**: Transaction can release locks but cannot acquire any new locks.

Diagram:



Advantage: Ensures serializability.

Disadvantage: May cause deadlock.

5. When a schedule is called non-recoverable schedule? Explain.

A schedule is called **Non-Recoverable** if it allows a transaction to commit even when a transaction that it read from (dirty read) has not yet committed, and later that transaction fails.

Condition for Non-Recoverable Schedule:

If transaction Tj reads a data item written by Ti, then the commit of Ti must occur before the commit of Tj.

Example of Non-Recoverable Schedule:

- T1: W(A)
- T2: R(A)
- T2: COMMIT
- T1: ROLLBACK

Here, T2 has read dirty data from T1 and committed. If T1 rolls back, the database becomes inconsistent. This is **non-recoverable**.

6. Differentiate between Serial Schedule and serializable schedule. Explain different types of locks.

Difference between Serial Schedule and Serializable Schedule:

Feature	Serial Schedule	Serializable Schedule

Execution	Transactions run one after another	Transactions are interleaved
Performance	Low throughput	High throughput
Correctness	Always correct	Equivalent to some serial schedule
Practical Use	Rarely used	Used in concurrent systems

Types of Locks:

1. **Shared Lock (S-Lock / Read Lock)**: Allows multiple transactions to read but not write.
2. **Exclusive Lock (X-Lock / Write Lock)**: Allows only one transaction to read and write. No other transaction can access the item.
3. **Update Lock (U-Lock)**: Used to avoid deadlock while upgrading from shared to exclusive lock.

These locks are fundamental for concurrency control in DBMS.

10 MARKS

1. Explain the ACID properties of transaction with example.

- **Atomicity:** The "all-or-nothing" execution property. Either all operations of a transaction succeed and are written permanently, or the database reverts to its original state if any operation fails.
- **Consistency:** A transaction must transform the database from one valid, logically consistent state to another, maintaining all predefined structural constraints and rules.
- **Isolation:** Concurrently executing transactions must remain completely unaware of and unaffected by each other's intermediate data modifications until they commit.
- **Durability:** Once a transaction commits successfully, its modifications endure permanently within non-volatile storage, surviving subsequent hardware failures or system crashes.

Example: Consider a bank funds transfer of ₹500 from Account A (initial balance: ₹1000) to Account B (initial balance: ₹2000):

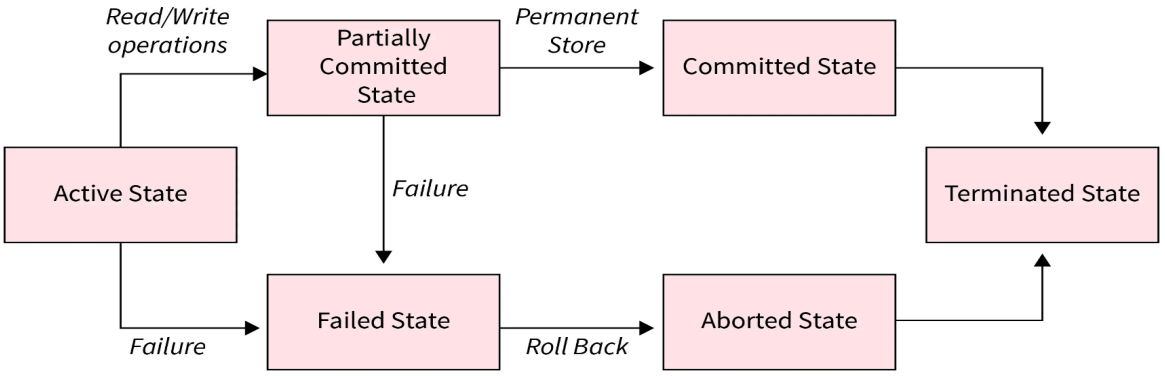
1. `Read(A)`, `A = A - 500`, `Write(A)`
2. `Read(B)`, `B = B + 500`, `Write(B)`

- **Atomicity:** If the system crashes right after `Write(A)` but before `Write(B)`, the database rolls back the change to A, ensuring no funds disappear.
- **Consistency:** The total combined sum before the transaction (₹1000 + ₹2000 = ₹3000) matches the sum after completion (₹500 + ₹2500 = ₹3000).
- **Isolation:** A concurrent transaction checking the balance of Account B midway cannot read its uncommitted intermediate state.
- **Durability:** After the confirmation message, the new balances (A=500, B=2500) remain saved even during immediate power loss.

2. Discuss the states of transaction with suitable diagram.

- **Active State:** The entry state where a transaction resides while executing its operational statement instructions (`Read`/`Write`).

- **Partially Committed State:** Reached instantly after the final operational instruction completes execution, before structural disk writing occurs.
- **Committed State:** Reached when the database updates are written to the physical storage log, making changes permanent.
- **Failed State:** Entered if the transaction encounters unexpected runtime runtime errors, software bugs, or hardware issues preventing normal completion.
- **Aborted State:** Reached after a failed transaction rolls back all intermediate operations, restoring data to its pre-execution state.



3. Explain Lock-based protocols in details.

Lock Modes:

1. **Shared Lock (S):** Requested for reading operations. Multiple concurrent transactions can simultaneously hold shared locks on a single data item.
2. **Exclusive Lock (X):** Requested for writing/modifying operations. Only a single transaction can hold an exclusive lock on an item, blocking all other lock requests.

Compatibility Matrix:

Requested \ Held	**Shared (S)**	**Exclusive (X)**
Shared (S)	Compatible	Incompatible
Exclusive (X)	Incompatible	Incompatible

Protocol Variants:

- **Basic Two-Phase Locking (2PL):** Guarantees conflict serializability by requiring transactions to lock items in two separate, non-overlapping chronological intervals:
    1. *Growing Phase:* The transaction requests and acquires various locks but cannot release any.
    2. *Shrinking Phase:* The transaction releases its accumulated locks but cannot request any new ones.
- **Strict 2PL:** Mandates the growing phase rule, but additionally requires all acquired **Exclusive (X) locks** to be held continuously until final execution completion (Commit/Abort). This eliminates cascading rollbacks.
- **Rigorous 2PL:** The most restrictive form; requires **all locks** (both Shared and Exclusive) to be held until the final transaction termination phase.

4. Draw the precedence graph for the schedule wl (A) w2 (A) r3 (B) w3 (B) r1 (B): S:{r1(x);r2(y);r3(y);w2(y);w1(x);w3(x);r2(x);w2(x)} Test whether the

schedule is conflict serializable or not? If Yes, then what is the correct order of serialization.